Privacy-Preserving Compliance

by

Joss Duff

A Thesis
Presented to the Graduate and Research Committee
of Lehigh University
in Candidacy for the Degree of
Master of Science

in
Computer Science

Lehigh University

May 2026

© 2026 Copyright
Joss Duff

Thesis Approval Signature Sheet

Thesis is accepted and approved in partial fulfillment of the requirements for the Master of Science in Computer Science.

Privacy Preserving Compliance
Joss Duff

_____
Date Approved

_____
Henry F. Korth, Thesis Advisor

_____
Brian Davidson, Department Chair

Acknowledgments

TODO

Table of Contents

Acknowledgments **iv**

Abstract **1**

1 Background and Definitions **2**
1.1 Zero Knowledge Proof . . . 2
1.2 Compliance . . . 3
1.3 Blockchain Background . . . 3
1.4 Privacy in the Blockchain Setting . . . 5

2 Problem **8**
2.1 Industry Interest . . . 9

3 Desired privacy preserving compliance properties **10**
3.1 No deanonymization . . . 10
3.2 Proactive compliance . . . 11
3.3 Rich compliance language . . . 11
3.4 Multiple compliance . . . 12
3.5 Secret inputs . . . 12
3.6 Verifiable compliance . . . 12
3.7 Chain agnostic . . . 13
3.8 Modular privacy . . . 13

4 Related Work and Attempted Solutions **14**

5 Proposed Solution **17**
5.1 Framework Approach . . . 17
5.2 Actors . . . 18
5.3 Composability . . . 20
5.4 Constraints . . . 21
5.5 Compliance definition contract . . . 26
5.6 Generating Compliance Proofs . . . 27
5.7 Proof Manager . . . 28
5.8 Example integrating application: Compliant Stablecoin . . . 29
5.9 Updating Compliance Definition . . . 31
5.10 Security Considerations . . . 33

6 Prototype Implementation **35**
6.1 Prototype Demo . . . 35
6.2 Benchmarks . . . 36
6.3 `MEM` constraint . . . 37

6.4 NON-MEM constraint . . . 37

7 Future Work **39**
7.1 Constraint Domain-Specific Language . . . 39
7.2 Proof inference and optimization . . . 40
7.3 Backend provers . . . 41
7.4 Carry compliance proof through mixers . . . 41
7.5 Alternative Applications of Constraint Proving . . . 43
7.6 Latency Optimization . . . 43
7.7 Industrial and Regulator Acceptance . . . 43

Bibliography **44**

A Appendix: Framework Specification **51**
A.1 Terminology . . . 51
A.2 On-Chain Contracts . . . 51
A.3 Proof System Requirements . . . 56
A.4 Off-Chain Artifact Storage . . . 58
A.5 Interaction Protocols . . . 60

B Appendix: Example Constraints **64**
B.1 Non-Membership Constraint (NON-MEM) . . . 64
B.2 Membership Constraint (MEM) . . . 64
B.3 Protocol Interaction Constraint (INTERACT) . . . 65
B.4 Protocol Avoidance Constraint (AVOID) . . . 65
B.5 Age Constraint (AGE) . . . 65
B.6 Structuring Constraint (STRUCTURE) . . . 65

Vita **67**

List of Tables

4.1 Comparison of privacy preserving compliance solutions 16

6.1 MEM constraint benchmarks 37
6.2 NON-MEM constraint benchmarks 38

List of Figures

5.1 Composable compliance framework overview 20
5.2 Multi-compliance example 21
5.3 Proof aggregation workflow 27
5.4 Proof Manager architecture 29

Abstract

Privacy and compliance in blockchain systems face an inherent contradiction: privacy requires that only the individual knows certain information, while compliance traditionally requires revealing information to prove compliance criteria. This tension has led to two unsatisfactory outcomes:

1. Privacy systems without compliance attract illicit actors and face legal challenges.
2. Compliance systems without privacy expose all transaction histories publicly, making them unacceptable for consumer and commercial adoption.

Neither outcome supports the widespread adoption of blockchain technology, particularly among traditional financial institutions where regulatory uncertainty and privacy concerns are the top barriers to entry.

We present a composable framework that resolves this conflict by enabling users to prove compliance cryptographically without revealing private transaction data. In our system, regulatory bodies publish compliance definitions as combinations of reusable logical primitives called constraints. Applications select relevant compliance definitions based on their regulatory obligations. Users generate zero-knowledge proofs demonstrating that they meet these requirements, which are then verified on-chain before transactions execute. This provides proactive compliance: preventing non-compliant actions rather than detecting them afterward, all while maintaining strong privacy guarantees.

Our framework is distinguished by its composability and generality, designed explicitly for a global ecosystem with many regulatory bodies whose requirements evolve over time. Compliance definitions are chain-agnostic, modular with respect to privacy mechanisms, versioned to accommodate regulatory updates, and can be combined arbitrarily which allows applications to satisfy multiple jurisdictions simultaneously.

1. Background and Definitions

Below is relevant background information on blockchains, blockchain privacy systems, ZK proofs, and our working definition of "compliance". People familiar with blockchain and ZK can skip this section.

1.1 Zero Knowledge Proof

Zero Knowledge (ZK) proofs are a cryptographic technique that allows a party to prove specific claims about a computation without revealing any additional information about the computation. The origins of this concept predate blockchain [7, 27], but it was described elegantly and simply by Vitalik Buterin et al. in [14] as follows:

> The "claim" proven by a ZK proof is expressed as a type of program that is often called a *circuit*. Mathematically, it suffices to think of it as a function $f(x, w) \rightarrow \{\text{True}, \text{False}\}$, where x is the public input, w is the private input, and $f(\cdot)$ is the function being computed. A ZK proof proves that, for a given x known both by the prover and the verifier, the prover knows a w such that $f(x, w)$ returns True.

SNARKs are a specific type of ZK proof that has the unique property of *succinctness*: The proof size doesn't grow with the size of the computation, and the proof can be verified very quickly relative to the computation [46].[1]

A drawback with some SNARK schemes, such as Groth16 [28], is that they rely on a *trusted setup*: a time-intensive ceremony involving multiple participants. A trusted setup needs to be performed once per circuit [35] for Groth16 and some other SNARKs.

[1] The SNARK acronym comes from Succinct, Non-interactive ARguments of Knowledge. The non-interactive term serves to distinguish SNARKs from the interactive proofs between a prover and a verifier that are often used to illustrate the concept of ZK for a general audience.

A type of SNARK proofs, PLONK, mitigates this drawback by allowing trusted setups to be re-used across circuits. With PLONK proofs, it is not necessary to initiate a new trusted setup for every new circuit [26].

1.2 Compliance

Compliance is referred to as the goals and properties of a system to attempt to adhere to a law or policy. Compliance isn't immunity from prosecution, it is a best effort to prove legality of actions. For example, US companies have systems in place to prevent money laundering in order to be compliant with the US Bank Secrecy Act (BSA) - also referred to as Anti-Money Laundering (AML) [62]. A typical effort to comply with AML in the blockchain space is blocking addresses associated with sanctioned individuals from interacting with a protocol.

A goal of this framework is to be flexible enough to accommodate compliance definitions over all aspects of a transaction: the payer, the payee, the content of the transaction, and even the actors' transaction histories. Flexibility results in the ability to express any definition of compliance. This framework is built around the fact that definitions of compliance vary over time and by jurisdiction.

1.3 Blockchain Background

Blockchains are fundamentally data-storage systems with unique properties. Most importantly, data stored on a blockchain (on-chain) are immutable: history cannot be changed. This property is achieved by allowing anyone around the world to participate in consensus and validation of the blockchain's state by running a node. There is a small class of *permissioned* blockchains with access restrictions on who can use the chain or run a node.

Nodes have a financial incentive to participate correctly in consensus. This mechanism makes it economically infeasible for a single attacker to disrupt the network. For example, in the Ethereum network, nodes who want to participate in block production must put up a stake of the Ether currency[2] which is *"slashed"* (lost forever) if the node behaves incorrectly[53].

[2]The original staking requirement was 32 Ether but it is subject to change.

The first blockchain, Bitcoin, was created in 2008 and proposed a trustless ledger of transaction histories[41]. Ethereum was created in 2013 and improved on Bitcoin by allowing trustless execution of Turing-complete logic on-chain, referred to as *smart-contracts* [12]. This allowed for more complex applications like marketplaces and lending platforms to exist in a decentralized, permissionless, and immutable setting. These on-chain applications are sometimes referred to as "decentralized applications" or *dapps* for short.

Every transaction on a blockchain has a non-static monetary cost paid to the block-producing node as an incentive to include the transaction in the next write. Some blockchains with smart-contract support, like Ethereum, include an additional transaction cost driven by the complexity of the smart-contract logic being executed.

Users on blockchains are represented by a public-key address. The user signs messages and transactions with the associated private key. Users can have multiple public-key addresses. It is impossible to reveal who is associated with a public key unless the users identity is revealed. Identity can be revealed directly by the user or indirectly by others who know the public key's identity, like a centralized exchange.

A centralized exchange (CEX) is a company that allows users to trade traditional currency for cryptocurrency. They often require the user to connect their bank account and supply regulatory documentation such as identification cards and proof of residency. They collect this information to comply with "Know Your Customer" (KYC) laws. Using a CEX is a common method for acquiring cryptocurrency. CEXs are *custodial*: they control the address (they know the private key) that owns the cryptocurrency on-chain and allow the user the ability to trade or transfer the cryptocurrency. They also allow users to send their cryptocurrency from the CEX account to the users' own on-chain address. CEX accounts are not addresses. Rather, the CEX keeps its own addresses and the user's holdings are only an accounting record in the CEX's books.

A decentralized exchange (DEX) enables trading cryptocurrency while letting users retain direct control of their holdings. In its simplest form, the DEX creates a smart-contract to implement the trade and the users sign it with their private key.

1.4 Privacy in the Blockchain Setting

All data on blockchains are public by default unless the chain supports special privacy features like Aztec Network [67] or Zcash [8]. There are also some enterprise permissioned blockchains that restrict visibility levels. However, the vast majority of chains are public by default, including Ethereum and Bitcoin.

For public blockchains, every user's entire transaction history is exposed to the world. This is obviously undesirable for consumers. Most people don't want their purchase or subscription history visible to the entire world. Many applications and protocols exist to provide users varying degrees of privacy.

Blockchain privacy protocols fall into two broad categories:

- *App-layer privacy*: Applications on top of existing blockchains.
- *Base-layer privacy*: Entirely new blockchain architectures with privacy as a core identity.

1.4.1 App-layer privacy

CoinJoin was the first permissionless app-layer transaction mixer [37]. CoinJoin was built on Bitcoin and combined multiple input Unspent Transaction Outputs (UTXOs) from different parties into multiple outputs, obscuring which inputs correspond to which outputs. Tornado Cash was the first transaction mixer to use ZK proofs [45], and saw massive popularity [22]. ZK proofs are a common tool used to achieve privacy; ZK allows a party to prove a computation without revealing the entire computation. See section 1.1 for an introduction to ZK proofs. The black-hat group Lazarus used Tornado Cash to aid in laundering stolen funds and the Tornado Cash developers were put on trial [63]. Railgun [51] and Hinkal [29] are transaction mixers that focuses on compliance. Privacy Pools is a recent transaction mixer that also focuses on compliance with *association sets* [14]. Ultra-Anon is an experimental privacy token that has the property of *plausible deniability*: it isn't revealed if users utilized privacy [24]. Stealth addresses are ephemeral public keys that can receive any tokens while hiding the receiver address [13]. zkBob is a wallet that has private transfer for certain stablecoins [69]. Webb Protocol creates a cross-chain anonymity set [58]. Worm is

an application implementation of "EIP-7503: Zero-Knowledge Wormholes" [33], which is another transactional privacy solution with the property of plausible deniability.

1.4.2 Base-layer privacy

Zcash is a privacy blockchain based on Bitcoin's UTXO model. Zcash is implemented from the 2014 Zerocash paper [8]. Monero is another private blockchain that leverages the UTXO system and is based on the 2013 CryptoNote paper [64]. Grin is an implementation of the 2016 Mimblewimble protocol, a modification of Bitcoin with private transfers [30]. Aztec is a general purpose layer 2 [3] blockchain that allows for private state with smart-contract support [67]. Aleo [2], based on the 2018 paper "ZEXE: Enabling Decentralized Private Computation" [10], is also a blockchain with private state and smart-contracts. The Miden blockchain supports the private state similarly [48]. Penumbra is a private network and decentralized exchange for the Cosmos ecosystem [44]. Arc [36] and Tempo [59], founded by Circle and Stripe, respectively, are stablecoin-focused blockchains with some degree of transaction privacy. Payy is an Ethereum layer 2 with native transaction privacy and compliance mechanisms [38]. Secret Network has private by default smart contract state [55]. Ernst & Young (EY) released a permissioned layer 2 blockchain, Nightfall, with private transactions and compliance features [25].

1.4.3 Custodial mixers

Custodial mixers are transaction privacy solutions that require users to send funds to a trusted intermediary who then redistributes the funds. This accomplishes the privacy goal of breaking the on-chain traceability between the input and output transactions, but custodial mixers have the major weakness that operators could steal funds or keep records. This is fundamentally weaker privacy because that intermediary has knowledge of a user's transaction history, rather than that history being private and held only by the user. While this is not a legal document, we note that in some jurisdictions, the intermediary could be subject to government subpoena.

Blender.io was an example of a custodial mixer on Bitcoin where users sent funds to a

[3] A layer 2 blockchain periodically syncs it state to another blockchain, referred to as the layer 1. In doing so, the layer 2 blockchain inherits the security and data guarantees of the layer 1.

trusted operator who then re-distributed the funds. Blender was sanctioned in 2022 by the U.S. Treasury Department for failing to meet AML regulations [60].

Some centralized exchanges can also be used as custodial mixers. A user deposits into the centralized exchange from one address and withdraws from another address. From the on-chain perspective, this looks like two unrelated transactions among the millions that occur within that centralized exchange. However, the centralized exchange has full information and knows the two addresses are owned by the same user.

1.4.4 Anonymity Set

An *anonymity set* is the group of entities among which an observer cannot reliably identify which specific entity possesses a particular attribute or performed a particular action [47, 50]. Anonymity scales with set size: the larger the anonymity set, the less identifiable each entity is [23].

For transaction mixer applications on blockchains like Railgun or Tornado Cash, any address that has deposited can withdraw, but it is never revealed which depositing addresses have an associated withdrawal. Each withdrawal could have come from any of the depositing addresses. Therefore, the anonymity set of any given withdrawal from the mixer is the set of all addresses that have deposited in the mixer. This is oversimplified, and in practice anonymity sets can be reduced with chain analysis [32, 39]. Anonymity here is referring to the obfuscation of the link between a depositing and withdrawing address.

2. Problem

Blockchains are fundamentally pseudonymous and public: user identities are obfuscated behind addresses, but all activity between addresses is public by default. In practice, entities are able to associate identities to addresses when on-ramping traditional funds to cryptocurrency and therefore track individuals' flows of funds on-chain. In order for cryptocurrency to be practical for consumers, it must at the very least have the same level of privacy as traditional finance, but ideally improves on traditional finance privacy. In traditional finance, typically two entities know information about a user's funds; the user and the exchange that is custodian of the funds. In blockchain settings, all entities can track user funds and activity. Neither consumers nor corporations will want their entire trading, purchasing, or subscription histories publicly available.

Privacy is a growing field in the blockchain space [45, 33, 9, 24], but raises the concern that malicious actors may use privacy to launder funds. In an effort to reduce money laundering, regulatory bodies are pursuing legal action against developers of privacy protocols [34, 63, 19]. This introduces the need to explicitly meet regulatory compliance inside private on-chain applications.

The goal is to maximize privacy while maintaining the ability to prove compliance. Current research efforts attempting to bridge privacy and compliance have proposed solutions with strict domain limitations such as a new blockchain architecture, specific on-chain protocols, or privacy violations. We investigate a framework that allows for compliance in any blockchain smart-contract environment and in any privacy protocol without compromising privacy provided by the protocol.

2.1 Industry Interest

The two privacy protocols currently operational that have some measure of compliance are Privacy Pools and Railgun. Both have received a great deal of interest from users; Railgun was launched in 2022 and reached $132 Million (USD) deposited [21] in August 2025. Privacy Pools launched in March of 2025 and reached $3.3 Million (USD) deposited [20] in January 2026. Many users currently in the blockchain space desire both compliance and privacy.

Large traditional financial corporations are yet to integrate significantly with blockchains. In a 2025 survey, 300 major financial firms were asked what factors keep them from becoming more involved in blockchain technology. In this multiple-choice question, 52% cited lack of regulatory certainty and 50% cited lack of privacy as factors [43]. These are two of the top three blocking factors. We see it is not just users, but also institutions who desire compliance and privacy. Solving this would result in a significant amount of capital being allocated on-chain.

In 2024, $2.2 billion (USD) was stolen in crypto hacks and exploits [61]. Tracing stolen funds from all hacks is too large a task, so auditors prioritize only the largest exploits. Our framework provides strong **proactive** compliance guarantees. Non-compliant actors are stopped from interaction with compliant applications. Compare this to retroactive compliance measures where non-compliant actors are able to interact with private protocols freely, but there exists a key that is able do decrypt any user's transaction history to be used by law enforcement. This retroactive approach is sometimes referred to as view-key compliance. Not only does this create more work for auditors trying to trace non-compliant individuals, it also introduces the risk of leaking compliant user's private information.

3. Desired privacy preserving compliance properties

The following properties provide a path for a strong and flexible compliance solution without compromising the benefits of having a privacy system on-chain. The best on-chain compliance system will have no deanonymization, proactive compliance, a rich compliance language that allows for multiple compliance definitions, secrets inputs, the ability to verify compliance status, and no restriction on blockchain architecture or privacy mechanism.

3.1 No deanonymization

Deanonymization refers to revealing private information about an address, like balances or transaction history. No entity should have the ability to deanonymize user data in an on-chain privacy system[1].

Any compliant privacy solution that gives privileges to decrypt private user data has deanonymization. The ability for a third party to reveal a user's private details is fundamentally weaker privacy. We can guarantee that private transactions can only be revealed by the user, and we provide this guarantee without compromising compliance. We also provide stronger compliance guarantees than view-key compliance. Viewing keys are used for retroactive compliance checks, and our framework has proactive compliance: non-compliant actors are stopped before making their transaction rather than after. Additionally, with view-key compliance a user can never verify if their secrets have been read or shared. In our solution a user's secrets are always kept private.

The existence of the ability to deanonymize users is a risk vector to users and compromises trustlessness, as highlighted in [11]. Individuals with access to private user information can and will abuse it. Examples include the case of Twitter employees selling personal user

[1] Caveat: A user can always choose to reveal their own private transaction history. This is acceptable because the user is initiating their own deanonymization. "No deanonymization" refers to third-party deanonymization.

data to Saudi Arabia [42, 17], or US telecom providers illegally selling user data [18]. Additionally, any centralized collection of user data can be hacked. In 2024, data that US telecommunication companies were legally required to collect was hacked [54]. In 2025, personal data held by the Ukrainian government was hacked by Russia [49]. In 2022, 23 terabytes of personal information on 1 billion Chinese residents was leaked from a police database and sold [66].

On-chain privacy is unique because it allows for truly private-to-the-world transactions. An on-chain privacy system where a third party also knows secret user data is as "private" as a traditional exchange or bank. In order for a on-chain privacy system to provide unique value to users, it must not allow for deanonymization.

3.2 Proactive compliance

A compliance system is *proactive* if it prevents non-compliant action rather than detecting it afterwards. A proactive compliance system both preserves user privacy by eliminating the need for information disclosure and reduces regulatory burden by automating compliance checks. Regulators can check compliance by verification of proofs, which is much more efficient than having to evaluate full execution histories. Users can show transparently that they are in compliance without having to reveal private data. Our framework prevents non-compliant actions by requiring a proof of compliance before a transaction is executed.

3.3 Rich compliance language

Compliance can be expressed in many languages. At the lowest level, it is the language of legal documents. Expressing compliance on-chain is more restrictive because of the inherent setting. Many protocols express compliance on-chain as only a set of sanctioned addresses. We allow for the maximum expression of compliance by allowing any on-chain data to be used in a compliance definition: transaction histories, contract states, fund origins, user secrets, etc. A rich compliance language allows for regulators to clearly define exactly what actions are non-compliant, leading to less ambiguity and requiring the regulators to decide on a concrete definition.

Additionally, we include time as a first class parameter compliance definitions. This

captures the invariant of real-world regulations changing over time. The time parameters of compliance definitions are accessible on-chain: users are able to prove that they were compliant with a regulation at a previous time and the history of a compliance definition is transparent.

3.4 Multiple compliance

There are many government bodies in the world, each with their own laws and regulations. A suitable compliance protocol should be able to handle any amount of compliance definitions. A compliance protocol that only supports one government's regulations is not suitable for an international finance system.

3.5 Secret inputs

We allow for inputs in the compliance evaluation that are only known to the user. This is trivial in most solutions that utilize ZK proofs for compliance verification, as ZK proofs inherently allow secret inputs. However this is difficult to achieve in systems where the compliance evaluation is done on a remote server, and impossible in systems with deanonymization.

3.6 Verifiable compliance

Compliance criteria should be publicly viewable and a user's compliance status should be verifiable. A user's compliance status should not be determined behind a third-party API as this runs the risk of the third-party not properly enforcing the compliance definition, introducing risk to all parties involved. This trust assumption could potentially be mitigated with techniques such as *trusted execution environments* (TEEs) that verify that a third-party correctly executed some computation [5]. The approach in this framework is to publish compliance definitions publicly and have users generate their own ZK proofs that can be cryptographically verified.

3.7 Chain agnostic

A compliant privacy solution is *chain agnostic* if it does not rely on any protocol level mechanism of specific blockchain. Application layer solutions are chain agnostic by default because they exist on-chain as a set of smart contracts.

3.8 Modular privacy

Some compliant privacy solutions are dependent on a specific implementation of on-chain privacy. Privacy preserving compliance shouldn't depend on privacy mechanisms, as new privacy mechanisms are constantly being researched. A compliant privacy solution has *modular privacy* if it can be applied to any privacy protocol regardless of its mechanism.

4. Related Work and Attempted Solutions

ZEXE [10] mentions the ability to prove non-membership of a list of sanctioned addresses but the implementation, Aleo [2], utilizes selective disclosure view keys that allow the user to reveal a subset of their transaction history to a 3rd party.

Payy [38] attaches a transaction lineage to each UTXO, but also mentions the potential for a regulator-based ZK compliance mechanism like this paper presents in detail. With transaction lineage, some privacy is lost because a receiver of a note is able to view previous holders.

Arc [36] allows institutions to have full visibility of their customers on-chain as well as selective disclosure view keys.

Tempo [59] has opt-in compliance policies managed by outside actors, similar to our regulator model (Section 5.2.2), but token issuers always maintain full visibility.

"A privacy-preserving scheme with multi-level regulation compliance for blockchain" [31] (referred to as "ML Reg" below) has multiple levels of view keys with each level giving more visibility than the last.

REGKYC [68] proposes that centralized exchanges provide users signatures of arbitrary KYC attributes. The signatures can be verified on-chain, allowing a specific privacy implementation, "PRIVKYC", to be gated by certain KYC attributes. It also has a mechanism to deanonymize users under certain conditions.

Haze and Daze [4] are two variations of a compliant privacy mixer. Both allow for non-sanctioned addresses to deposit, but Daze allows for deanonymization under certain conditions.

ZEBRA [52] is an anonymous credential scheme for on-chain access control where a certificate authority issues credentials with arbitrary attributes. It allows a quorum of auditors to deanonymize misbehaving wallets and revoke all wallets associated with a credential.

SeDe [16] proposes modifying UTXO notes with auxillary data, allowing a revoker to recursively deanonymize an illicit transaction subgraph by decrypting notes after a threshold quorum of guardians approves the request.

"A Regulation Scheme Based on the Ciphertext-Policy Hierarchical Attribute-Based Encryption in Bitcoin System" [65] (referred to as CP HABE below), modifies Bitcoin's signature scheme to derive on-chain addresses from real-world identities. When an address is suspected of illicit activity, permissioned "regulator" nodes deanonymize the address and ban all addresses associated with that person.

Railgun [51] is a protocol that allows users to deposit into a transaction mixer with an optional proof that the user is not on a sanctions list. If a withdrawal address is suspected of illicit activity, Railgun can reveal the set of addresses that did indeed generate a proof on deposit, greatly reducing the anonymity of the illicit withdrawal address and making it trivial to link it to a depositing address.

Hinkal [29] is a protocol that allows enterprises to integrate on-chain while providing privacy and compliance techniques including selective disclosure viewing keys, deanonymization of addresses, and sanction list checks.

Privacy Pools [1] is based on the paper "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium" [14] and is a transaction mixer that requires a proof of non-membership on a sanction list to deposit. It supports multiple sanction lists in the same pool and allows users who proved against one list to dis-associate from users who proved against another.

Nightfall [25] requires institutions externally approved credentials for users on-chain.

Table 4.1: Comparison of privacy preserving compliance solutions

Solution	**No deanonymization**	**Proactive compliance**	**Rich compliance language**	**Multiple compliance**	**Secret inputs**	**Verifiable compliance**	**Chain agnostic**	**Modular privacy**
Aleo [2] (ZEXE [10])	✗	✗	✗	✓	✗	✗	✗	✓
Payy [38]	✗	✗	✗	✗	✗	✗	✗	✓
Arc [36]	✗	✗	✗	✗	✗	✗	✗	✓
Tempo [59]	✗	✓	✓	✓	✗	✓	✗	✓
ML Reg [31]	✗	✗	✗	✗	✗	✗	✗	✓
REGKYC [68]	✗	✓	✓	✓	✓	✓	✓	✗
Daze [4]	✗	✓	✗	✗	✗	✓	✓	✗
Haze [4]	✓	✓	✗	✗	✗	✓	✓	✗
ZEBRA [52]	✗	✗	✓	✗	✓	✓	✓	✓
SeDe [16]	✗	✗	✗	✗	✗	✗	✓	✗
CP HABE [65]	✗	✗	✗	✗	✗	✗	✗	✓
Railgun [51]	✓	✗	✗	✓	✗	✗	✓	✓
Hinkal [29]	✗	✓	✗	✗	✗	✗	✓	✗
Privacy Pools [1, 14]	✓	✓	✗	✓	✗	✓	✓	✓
Nightfall [25]	✗	✗	✗	✗	✗	✓	✗	✗
Our framework	✓	✓	✓	✓	✓	✓	✓	✓

5. Proposed Solution

We propose an open framework where regulatory bodies publish compliance definitions, applications require users to prove they meet those requirements, and users generate cryptographic proofs demonstrating compliance, all without revealing private information. Users who don't meet the compliance requirements are unable to interact with compliant applications. In this chapter we motivate the rational behind the framework and the high level description. For a detailed explanation on the framework specification, see appendix A.

Proving compliance to an application has been mentioned in brief in [11] and is live in Railgun and Privacy Pools. Our contribution is a standard framework to allow this compliance technique to be used at-scale across many applications and provides a standard interface for regulators to define compliance.

5.1 Framework Approach

The distinguishing feature of our solution relative to other work in this space is that it defines an open framework for use by players in a competitive ecosystem. It is not a new blockchain. It is not a payment system. It is not a new application. Rather, we are creating a framework for established and new-entrant financial providers to take advantage of the power of the emerging blockchain ecosystem by leveraging the features of our framework.

Our framework is unique in its provision of a standard means for regulators to publish requirements openly in a way that enables proof of compliance. Regulation based on constraints (Section 5.4) and proofs is valuable not only to financial providers, but also to regulators and users. Regulators can check compliance by verification of proofs, which is much more efficient than having to evaluate full execution histories. Users can show transparently that they are in compliance without having to reveal private data.

Our framework is *chain agnostic*: it can be implemented on any chain with smart-contracts and capability to verify ZK proofs. Because on-chain compliance pertains to transactions and transaction histories and not to specific proprietary features of a particular blockchain, our framework enables interoperability across platforms that use our framework. This facilitates a competitive ecosystem in which each member of the ecosystem shares a common foundation of privacy and compliance guarantees.

5.2 Actors

The actors in our framework consist of regulators, applications, and users. A simplified overview of the actor interaction:

1. *Regulators* create and publish *compliance definitions*.
2. *Applications* select relevant *compliance definitions* based on their regulatory obligations.
3. *Users* generate ZK proofs of *compliance definitions*, demonstrating that they meet requirements without revealing transaction histories, balances, or counterparties.
4. *Applications* verify proofs on-chain before allowing transactions, preventing non-compliant activity cryptographically rather than detecting it afterward.

5.2.1 Compliance Definition

Compliance definitions in this framework are sets of logical rules over blockchain data with the goal of isolating non-compliant actors (e.g., "address not on sanction list," "funds from KYC'd exchange," "no transaction-structuring patterns"). Compliance definitions contain public parameters, for example a list of sanctioned addresses, along with logical operations between those parameters. Compliance definitions are more technically described in section 5.4.3.

A compliance definition consists of a ZK circuit and an on-chain proof verifier contract.

Regulators (see below) create compliance definitions to enforce real-world regulation on-chain, for example a U.S. AML compliance definition.

Each published compliance definition can be used by any number of applications, and applications can require multiple compliance definitions. Each government in the world has its own laws and regulations, and some applications wish to be compliant with multiple regulations.

Compliance definitions are updatable by their author to account for changing regulations. For that reason, each compliance definition has an associated time parameter.

5.2.2 Regulators

Regulators create compliance definitions over on-chain data and publish these definitions publicly with a signature. The manner in which they are published is an implementation detail, but the compliance definitions need to be publicly accessible and in a form that can be compiled into a ZK circuit for users to prove.

Publishing a compliance definition is permissionless and we refer to any entity that does as a "regulator". Ideally, the entity is associated with the government regulatory agency, but in practice, particularly initially, the publisher is most likely to be a blockchain analytics company. These companies already assume compliance responsibilities, such as maintaining sanction lists and monitoring for suspicious chain activity.

As this is an open framework, there is no restriction on who can publish compliance definitions. Applications need to be careful to select definitions published by reputable sources.

Regulators are also responsible for updating compliance definitions, like adding more constraints or updating parameters, with each update having a new timestamp. Here, "update" of a contract means a parameter change as discussed in Section 5.9.

5.2.3 Applications

On-chain applications determine which compliance definition(s) suit their regulatory obligations.

Applications integrate with this framework by making a smart-contract change to require users to provide a proof of the required compliance at entry-point functions. If the proof does not verify, then the function reverts. After this change, it is guaranteed that all future

users of the application are compliant.

For example, the entrypoint function for some application is `deposit(someVars)`. Integration would consist of changing it to add parameters *proof* and the proof's required inputs *publicInputs*: `deposit(someVars, proof, publicInputs)`, and then beginning the function by using a **require** statement to mandate verification of the proof: `require(verifierContract.verify(proof, publicInputs))`.

5.2.4 Users

Users locally generate and store ZK proofs of compliance definitions and supply the proofs as an argument when interacting with a compliance requiring application. The constraint system was constructed to intelligently minimize the amount of proof computation needed for users interacting with many compliance requiring applications. Users are responsible for locally keeping previously generated proofs. This process is aided by the proof manager, discussed in section 5.7.

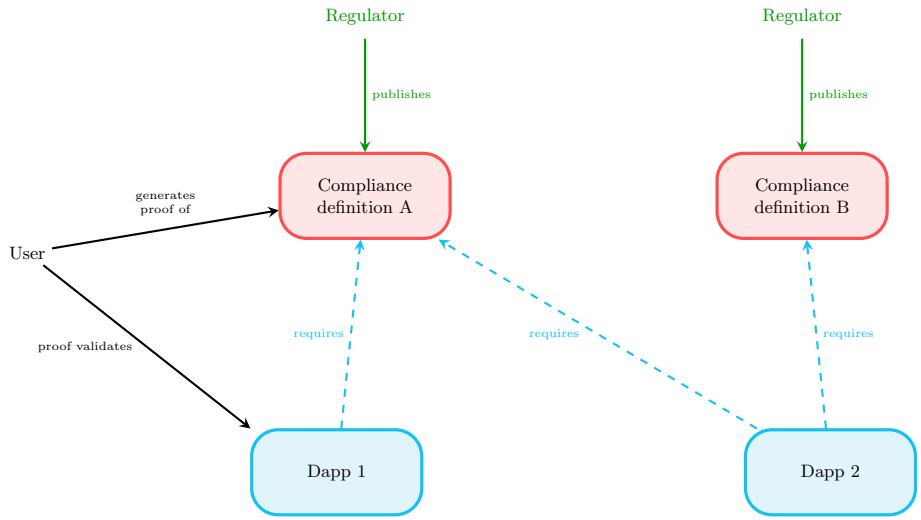


Figure 5.1: Composable compliance framework. Applications can opt into requiring multiple compliance definitions. Users can generate proofs of any compliance definition. In this example, a user is interacting with Dapp 1 so they must first generate a proof of compliance definition A.

5.3 Composability

Because compliance definitions are public, the same definition can be used by any number of protocols, a user can generate proofs for any published definitions of compliance, and proofs for multiple compliance definitions can be required by a protocol. Figure 5.1 shows

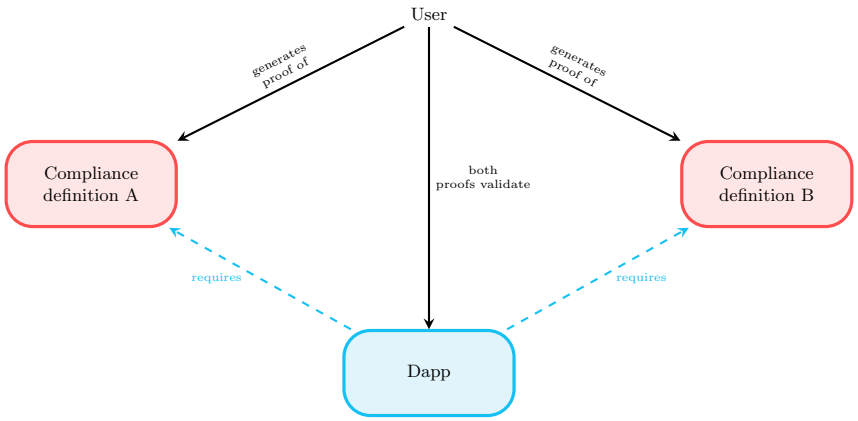


Figure 5.2: Example compliance scenario. A Dapp requires both A and B compliance. The user generates proofs for both compliance definitions and submits them together. The Dapp verifies both proofs on-chain before allowing the user to interact.

dapps opting into any number of compliance definitions. Figure 5.2 shows a user proving compliance for a dapp that requires multiple compliance definitions.

Compliance definitions can be fine-grained instead of wide-umbrella definitions. For example, instead of having a single definition that encompasses all US rules and regulations, there can be a definition for US consumer compliance and a definition for US corporate compliance.

The cost to verify a proof is independent of what is being proved. This is inherited from using zk-SNARKs as compliance proofs. The circuit can be very computationally intensive and have pages of criteria in its natural language form, but, although proof generation is computationally intensive, verification is not. Ease of verification enables efficient composability.

5.4 Constraints

Although regulations vary globally and over time, they often rely on similar logical primitives: checking membership in lists, comparing values against thresholds, or verifying historical patterns. We built our framework to leverage this similarity by decomposing compliance definitions into reusable logic that we refer to as *constraints*. This compositional structure allows for complex requirements to be expressed as combinations of constraints.

A *constraint* is a boolean predicate that evaluates blockchain state using first-order logic. A constraint C is expressed in the general form:

$$C = Q\, x \in D(s, i) : P(x, p)$$

where:

- **Quantifier** ($Q \in \{\forall, \exists, \ldots\}$): the logical quantifier
- **Chain State** (i): the blockchain state(s) from which the domain is evaluated. Chain states can be:
  - Atomic states:
    - Single block: B_k
    - Block range: $[B_i, B_j]$
  - State sets (with implicit universal quantification):
    - Parameterized windows: $\{[B_i, B_j] \subseteq [B_0, B_{\text{current}}] : \text{condition}\}$
- **Domain** ($D(s, i) \to \{x, \ldots\}$): a function that maps a subject s and chain state i to a collection of blockchain entities x over which the quantifier ranges. Common domains include:
  - Singleton: $D(s, i) = \{s\}$ evaluates a single subject (atomic case)
  - Transaction history: $D(\text{addr}, i) = \text{history}(\text{addr}, i)$
  - Set of recipients: $D(\text{addr}, i) = \text{recipients}(\text{addr}, i)$
  - Filtered collections: $D(s, i) = \{x \in D'(s, i) : \text{filter}(x)\}$
- **Subject** (s): the root blockchain entity from which the domain is derived. Example subjects include:
  - Sender of a transaction: sender(txn)
  - Recipient of a transaction: recipient(txn)
  - Account balance: balance(addr)
  - Result of a contract call: result(contract fn)
  - Transaction payload: payload(txn)
- **Parameter** (p): the value or set against which elements are evaluated, containing:

- Public parameters p_{pub} provided by the compliance author (e.g., sanction lists, required values)
- Private inputs p_{priv} provided by the user (e.g., mixer secret, identifying information)

- **Predicate ($P \to \{\textbf{true}, \textbf{false}\}$)**: a boolean function applied to domain elements. Predicates can operate on:
  - Individual elements ($P(x, p)$, where x is an element in the set returned by $D(s, i)$):
    - Comparison operations: $P(x, p) = (x \, \phi \, p)$ where $\phi \in \{=, \neq, <, \leq, >, \geq, \in, \notin, \ldots\}$
    - Nested constraints over the same chain state: $P(x, p) = C'(x, p', i')$
  - Entire collections ($P(X, p)$), where X is a subset of the set returned by $D(s, i)$:
    - Aggregate comparison: $P(X, p) = (\text{agg}(X) \, \phi \, p)$, where $\text{agg} \in \{\sum, \text{count}, \max, \min, \ldots\}$

5.4.1 Predicate constraints as special cases

Often regulators will want to make constraints independent of historic chain-state. These are useful for simple statements like "This address is not on sanction list A", where A is a public parameter available at the current chain state.

To simplify notation, we introduce a subclass of constraints called *predicate constraints*. Predicate constraints function exactly the same as the above constraint but with simplified syntax:

$$C = s \, \phi \, p$$

where $\phi \in \{=, \neq, <, \leq, >, \geq, \in, \notin, \ldots\}$, p is the parameter passed in, and s is the subject being evaluated. When predicate constraint notation is used, the chain state can be assumed to be the current block.

5.4.2 Example constraints

More example constraints can be found in appendix B.

Simple example: Sanction list checks are a popular compliance measure in the blockchain space [57, 40]. The constraint "Sender of this transaction is not on sanction list A" can be decomposed as the following predicate constraint:

- **Subject**: sender(txn) — the address that initiated the transaction
- **Operation**: $\notin$ — does not exist in
- **Parameter**: A — the sanction list (a public parameter provided by the regulator)

The constraint evaluates to **true** if and only if the subject (sender of the transaction) does not exist in the parameter (sanction list). Formally expressed as:

$$C = \text{sender(txn)} \notin A$$

Full example: The constraint "Throughout their transaction history, the sender of this transaction never interacted with a specific protocol" demonstrates the full constraint form. It can be decomposed as follows:

- **Quantifier**: $\forall$ — universal quantifier; the predicate must hold for all elements in the domain
- **Chain State**: $[B_0, B_{\text{current}}]$ — from genesis block to current block
- **Subject**: sender(txn) — the address that initiated the transaction
- **Domain**: txn_history(sender(txn), $[B_0, B_{\text{current}}]$) — all transactions sent by the subject within the chain state
- **Parameter**: p_{protocol} — the address of the prohibited protocol (a public parameter provided by the regulator)
- **Predicate**: recipient(t) $\neq p_{\text{protocol}}$ — the recipient of each historical transaction is not the prohibited protocol

The constraint evaluates to **true** if and only if every transaction in the sender's history was sent to an address other than the prohibited protocol. Formally expressed as:

$$C = \forall\, \mathrm{t} \in \mathrm{txn_history}(\mathrm{sender}(\mathrm{txn}), [B_0, B_{\mathrm{current}}]) : \mathrm{recipient}(\mathrm{t}) \neq p_{\mathrm{protocol}}$$

These constraints over historic chain state may seem expensive to prove but we examine techniques to reduce the amount of proof generation required in Section 5.6. Additionally, constraints proven once can be applied to multiple compliance definitions that reference those constraints 5.4.4.

5.4.3 Compliance definition in constraint language

A compliance definition is a combination of constraints with boolean operators and a time parameter during which the definition is valid. Given constraints $C_1, \ldots, C_k$, a compliance definition R is defined as:

$$R = C_1 \circ C_2 \circ \cdots \circ C_k, [t_{\mathrm{start}}, t_{\mathrm{end}}]$$

where $\circ$ denotes arbitrary boolean operators ($\wedge, \vee, \neg$, and parentheses for grouping), allowing for complex logical combinations of constraints. The time parameters t_{start} and t_{end} denote the timestamps defining the validity period of the compliance definition. The translation from a timestamp to a block number is an implementation detail. It is specified as a timestamp here because a constraint is independent of any specific chain. This temporal component is necessary to account for regulations changing over time and to allow for regulations to be published before they go into effect.

The definition R is considered active if and only if the evaluation occurs within this temporal window:

$$\mathrm{active}(R, t) = \begin{cases} \mathrm{true} & \text{if } t_{\mathrm{start}} \leq t \leq t_{\mathrm{end}} \\ \mathrm{false} & \text{otherwise} \end{cases}$$

Alternatively, open-ended validity can be expressed with $t_{\mathrm{end}} = \infty$ for rules with no expiration. For brevity, the time parameter is often omitted in this paper for compliance definitions and can be assumed to be valid for the current time.

In Future Work 7.1, we envision regulation being able to be compiled into constraints through an intermediary syntax, and constraints being able to compile into circuits.

5.4.4 Example: shared constraints

When multiple compliance definitions require the same constraint, that constraint can be proved once and applied to multiple definitions. For example, an application requires compliance definitions R_1 and R_2 which are made up of constraints C_1 and C_2:

$$R_1 = C_1 \wedge C_2$$

$$R_2 = C_1$$

A user of the application only needs to compute a proof for R_1 because its constraints are a super-set of the constraints used in R_2: proving R_1 also proves R_2.

Another Example An application requires R_1 and R_2:

$$R_1 = C_1 \wedge C_2$$

$$R_2 = C_2 \wedge C_3$$

Constructing a proof that validates for both regulations only requires generating a proof for three constraints, as C_2 is included in both definitions. C_2 is proved once and that proof is applied to both R_1 and R_2.

5.4.5 Off-chain data in constraints

Constraints can require off-chain data that is provided by the user during proof generation. This user input, p_{priv}, is completely private and not revealed to any other entity. This allows for constructing constraints that rely on off-chain data.

Public off-chain data, such as sanction lists, can also be used as constraints by writing the data on-chain.

5.5 Compliance definition contract

The `ComplianceDefinition` contract serves as the stable on-chain representation of a compliance definition. A compliance definition can have `ComplianceDefinition` contracts on multiple different chains. Applications reference this contract's address, which remains

constant even as the underlying verification logic is updated. For full details on the `ComplianceDefinition` contract structure, see A.2.3.

Each update to a compliance definition, whether a constraint update or a public parameter update, is recorded as a new entry.

This design provides several benefits: (1) a complete audit trail of all compliance updates, (2) the ability to verify proofs against historical definitions, and (3) support for publishing updates before they become active.

The verifier contract is an upgradable contract deployed and updated by its regulator and keeps a history of all compliance definitions it has previously verified for by referencing the addresses and IDs of each definition's corresponding verifier contract. The individual verifiers contain the time bounds t_{start} and t_{end} that annotate when that compliance definition is considered active.

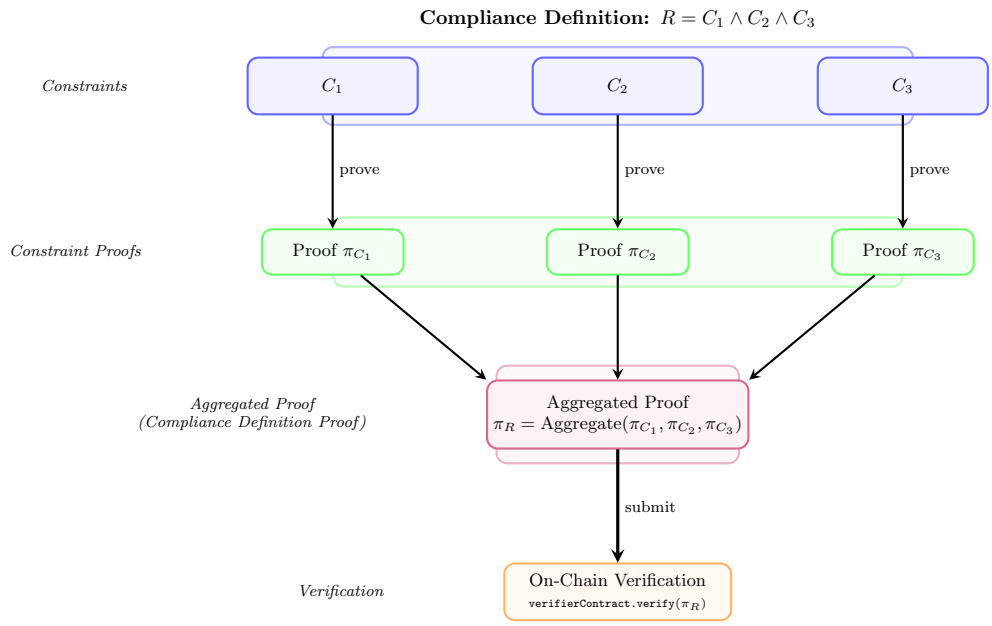


Figure 5.3: Proof aggregation workflow for compliance definition R. Individual constraints are proved separately, generating proofs π_{C_1}, π_{C_2}, and π_{C_3}. These proofs are then aggregated into a single proof π_R that is submitted on-chain for verification.

5.6 Generating Compliance Proofs

Proofs of compliance definition are aggregate proofs of multiple constraint proofs. When proving a compliance definition, each constraint is proved individually, then all constraint proofs are combined in an aggregation proof which is then submitted on-chain. See Fig-

ure 5.3 for the complete workflow.

Proofs are generated locally by the user and then submitted on-chain as transaction arguments. The proof is public on-chain[1], but it cannot be used to recover the user's private inputs.

The user can locally save all constraints that they have proven, discussed further in section 5.7. This allows users to re-use previous proved constraints. It is often the case that some parts of a compliance definition may already be proved or that the constraints of separate regulatory domains have some subexpressions in common. In these cases, inference over the constraints can lead to significant reduction in the complexity of the constraints to be proved, shown in section 5.4.4. Reducing the amount of proof generation necessary for a compliance definition via constraint inference is a promising future work direction 7.2.

Example Alice previously proves compliance definition R_1, which requires proving constraints C_1 and C_2. Alice has locally saved her proofs of C_1 and C_2.

$$R_1 = C_1 \wedge C_2$$

Now, Alice wishes to prove R_2.

$$R_2 = C_2 \wedge C_3$$

She can inspect R_2 and sees that it uses C_2, a constraint that she has previously proved and saved. She can skip the computation of C_2 because she already has a valid proof! All she needs to compute is C_3, and R_2, which is the aggregate proof over C_2 and C_3.

5.7 Proof Manager

We envision a client-side program that aids in proof generation as seen in figure 5.4. The proof manager takes as input a compliance definition, chain identifier, private user input, and the user address. It outputs a proof of that compliance definition.

The proof manager is responsible for

- Doing inference over required constraints to identify the minimum amount of proving necessary, relying on previously proved constraints. Minimizing proofs needed is more

[1]Some blockchains allow for private transactions. Proofs submitted as part of a private transaction will not be public.

efficient than speeding up proof generation.

- Constraint to circuit compilation.
- Fetching on-chain data of the user's transaction history necessary for proof generation.
- Fetching public proof inputs from the published verifier contract.
- Generating constraint proofs.
- Generating aggregate proof over the constraints to prove compliance.
- Saving completed proofs.

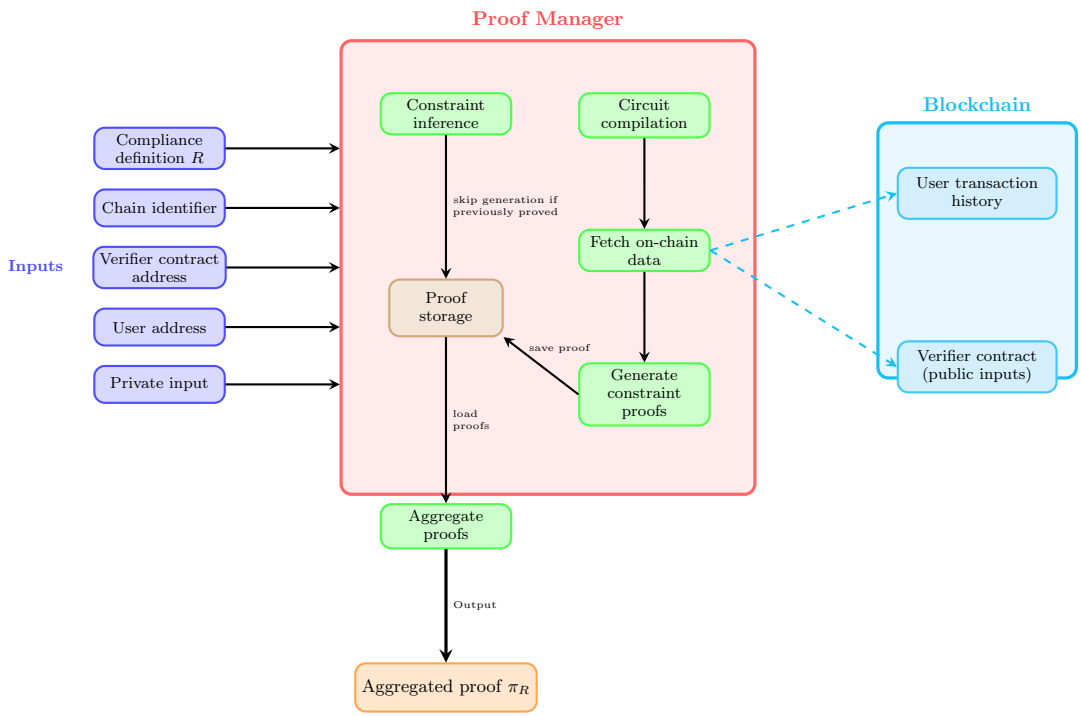


Figure 5.4: Proof Manager architecture. The system processes inputs through constraint inference (which checks proof storage for previously generated proofs), circuit compilation, fetching on-chain data from both the blockchain transaction history and verifier contract, and proof generation. Generated proofs are saved to proof storage and then loaded by the aggregation step to produce the final compliance proof π_R.

5.8 Example integrating application: Compliant Stablecoin

The author of this stablecoin contract is, for example, a US based company and wishes to only allow users who are not on a list of US sanctioned addresses $p_{\text{sanctioned}}$ to interact with this stablecoin.

The compliance definition requires both sender and recipient to satisfy the non-membership constraint:

$$R_{\text{stablecoin}} = C_{\text{non-mem}}(\text{sender}) \wedge C_{\text{non-mem}}(\text{recipient})$$

where both constraints evaluate against the same sanction list $p_{\text{sanctioned}}$.

A regulatory body deploys the $R_{\text{stablecoin}}$Verifier contract as well as publishing the circuit to generate proofs of $R_{\text{stablecoin}}$. The stablecoin contract author deploys the *CompliantStablecoin* contract and sets *verifierContract* to the address of $R_{\text{stablecoin}}$Verifier. Users who with to transfer *CompliantStablecoin* must first generate a proof of $R_{\text{stablecoin}}$ and supply that proof as an argument to the *Transfer* function.

```
contract CompliantStablecoin {
    // State
    mapping(address => uint256) public balance;
    ComplianceDefinition public complianceDefinition;  // R_stablecoin verifier

    // User calls transfer with normal arguments and the compliance proof they
        generated
    function transfer(
        address recipient,
        uint256 amount,
        bytes calldata proof,
        bytes32[] calldata publicInputs
    ) external returns (bool) {
        address sender = msg.sender;

        // Verify compliance through external verifier contract
        require(
            complianceDefinition.verify(sender, recipient, proof, publicInputs),
            "Compliance check failed"
        );

        // Execute transfer
        require(balance[sender] >= amount, "Insufficient balance");
        balance[sender] = balance[sender] - amount;
        balance[recipient] = balance[recipient] + amount;

        return true;
    }
}
```

This architecture demonstrates the composability of the framework: the `ComplianceDefinition` contract $R_{\text{stablecoin}}$ is maintained independently from the stablecoin logic, allowing the sanction list to be updated without modifying the stablecoin contract itself.

5.9 Updating Compliance Definition

Laws and regulations are constantly updated by governments; therefore, definitions of compliance will change over time. The framework allows updates to compliance definitions.

A compliance definition can be updated in two ways, and both cases involve the regulator changing the `ComplianceDefinition` contract state.

- Public parameter p_{pub} update (ex: add or remove an address from the sanction list) modifies the public inputs used in proof verification.
- Constraint update (ex: add or remove a constraint from the definition) modifies the circuit. A constraint update also requires regulators to publicly re-publish and sign the new compliance definition circuit.

Although our framework is independent of the specific prover used, there is a clear need to use a prover with either no trusted setup or a reusable trusted setup, for example

PLONK [26]. The number of distinct proofs that need to be computed in any transaction system using our framework makes it highly impractical to perform circuit-specific setup ceremonies. The options for handling updates are to deploy a new contract or update the existing contract.

- Regulator publishes a new verifier contract for the updated compliance and **all** applications must update their smart-contracts to reference this new contract address in order to remain compliant.
- Regulator updates their current verifier contract and no change is needed by the application.

We investigate both these approaches and find that verifier contract updating is better than deploying a new verifier contract.

If regulators publish a new verifier contract each time a sanction list is updated for a compliance definition, the applications using that compliance definition must manually update their smart contracts to reflect the new verifier contract address. As an example, Chainalysis, a leading blockchain analytics company used by government bodies, adds addresses to their on-chain sanctions list roughly every 30 days [15]. It is infeasible to require all applications to update their smart contract every 30 days. A priority of this framework is to provide applications with a strong guarantee of compliance and minimal overhead to the application.

For these reasons, we choose the path of regulators updating their existing compliance contract instead of publishing a new one on each update. This will keep the compliance verifier contract at the same address and does not require any action from the application. With this technique, applications have the guarantee that all their users meet the compliance definition with no windows of non-compliance.

5.9.1 Post Update Re-proving

Users who generated but did not use a compliance proof before the regulator updated the verifier contract may have to re-generate parts of their proof. They will have to regenerate the aggregate proof over the constraints, but might not have to re-prove all constraints

in that compliance definition. If a definition consists of constraints C_1 and C_2, and the regulator updates the public parameters of C_1, users will have to reprove C_1 but are able to re-use their proof of C_2.

Re-proving only the constraints that changed in an update is less costly than re-proving the entire compliance definition. Regulators have some incentive to make small updates since ease of proof generation leads to ease of compliance.

5.10 Security Considerations

5.10.1 Cross-chain Compliance Updates

If a compliance definition is updated across multiple independent blockchains, there can exist a small window of time after the update is finalized on one chain but before it is finalized on another. There is no native solution for atomic cross-chain transactions that accounts for different data layers and virtual machines.

The solution is to publish the updated compliance before it goes into effect. Recall that the compliance definition has time bounds:

$$R = C_1 \wedge C_2 \wedge \cdots \wedge C_k, [t_{\text{start}}, t_{\text{end}}]$$

Compliance definitions can be published before t_{start} time. If it is published far enough ahead of t_{start}, it is likely that the update transaction will be finalized on all chains by t_{start} when the update goes into effect. This is a natural consequence of making time a first-class parameter in compliance definitions.

5.10.2 Regulatory Arbitrage Window

There exists a brief period between when an address becomes non-compliant (e.g., gets sanctioned) and when the compliance definition's parameters are updated to reflect this, as discussed in [4, 14]. This window is inherent to any system that relies on parameter updates and is not unique to this framework.

Regulators should minimize this window by updating parameters promptly. Applications that require even stronger guarantees can implement a deposit delay that requires users to prove compliance for a period after deposit.

5.10.3 Proof Replay Prevention

A proof generated by one user must not be usable by another. Implementations bind proofs to the prover's identity by including the prover's address as a public input to the circuit. The mechanism for obtaining the prover's address on-chain is implementation-specific.

Without prover identity binding, an attacker could observe a compliant user's proof on-chain and replay it in their own transaction. This attack is prevented in the framework specification A.3.3.

5.10.4 Regulator Trust

Applications must exercise due diligence in selecting compliance definitions from reputable sources. A malicious regulator could:

- Publish an update that censors specific addresses.
- Publish incorrect parameters.
- Fail to update parameters promptly.

This framework does not prevent against malicious regulators, but it provides a solution. A key property of this framework is that publishing compliance definitions is **permissionless**. If an application is not satisfied with any of the available compliance definitions or untrusting of the regulators, they can easily publish and maintain their own definition. They can create a compliance definition from scratch or fork one of the publicly available definitions. However, "in-house" compliance results in more work for the application's team.

Additionally, because compliance definitions are public, any malicious update is also public. Transparent regulation is much safer for users and applications because the regulator must weigh the benefit of the malicious action against the consequences of the public fallout.

5.10.5 No Deanonymization Limitations

The framework guarantees that no entity can learn a user's private inputs from their proof.

However, metadata outside the framework (e.g., transaction timing, gas patterns, IP addresses, privacy mechanism of integration application) may reduce privacy. These concerns are outside the scope of this specification.

6. Prototype Implementation

This framework is applicable to any blockchain with a smart-contract language and constraints can be written in any syntax as long as they are representable in a ZK circuit. For our initial prototype implementation of this framework, constraints are expressed in the Noir ZK DSL [3] with the Barretenberg backend to generate PLONKish[1] proofs. The application integrating this framework will use the Ethereum Virtual Machine (EVM) expressed through the Solidity smart-contract DSL. Noir was chosen because it provides high-level language semantics for writing ZK circuits, but any ZK circuit language can be used. The EVM and Solidity were chosen because they have a large and polished ecosystem of smart-contract tools.

In the future, we envision a constraint compiler that compiles constraints from a formal syntax into custom circuits (see Future Work 7.1), but for prototype purposes, constraints are "hand compiled" into Noir circuits. Constraints are still listed to show the logical definition.

6.1 Prototype Demo

The implementation features a demo usage of this framework. In this demo environment, a regulator has previously created two compliance definitions: a sanction list check and a membership check, using the `NON-MEM` and `MEM` constraints, respectively. See below sections 6.3 and 6.4 for details and benchmarks of these constraints.

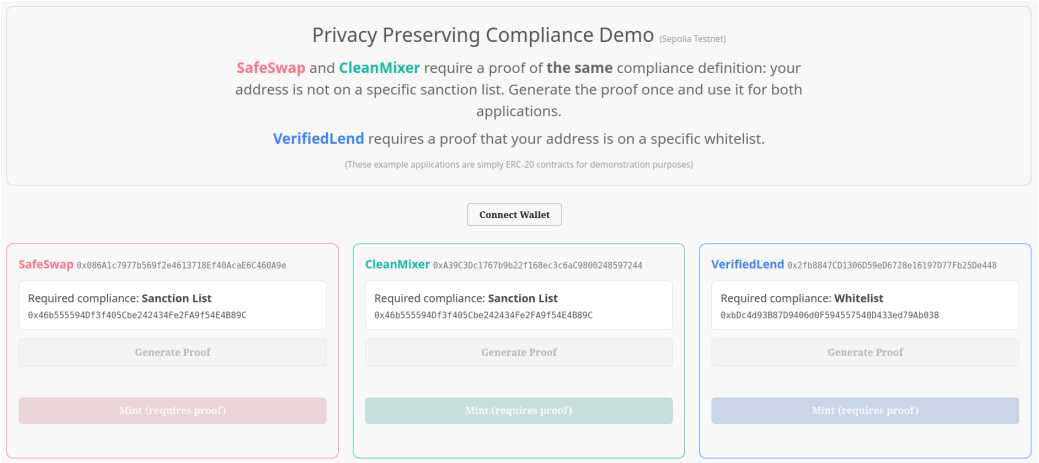
The demo has three example applications. "SafeSwap" and "CleanMixer" use the same sanction list compliance definition, while "VerifiedLend" uses a whitelist compliance definition. These applications are simply ERC-20 Token contracts for demonstration purposes.

The demo user connects their wallet and first must generate a proof of compliance before

[1] Specifically, we generate UltraHonk proofs, which is a variation of PLONK.

interacting with any of the applications. This proof is verified on-chain when the user calls the `Mint` function.

Notably, since SafeSwap and CleanMixer use the same compliance definition, a proof generated for one can be used by the other. If a user generates the sanction list proof for SafeSwap, they can *completely skip* the proof generation phase for CleanMixer. The proof verifies for both applications because it is a proof of the same compliance definition.



6.2 Benchmarks

Proof generation in this prototype implementation relies on Noir's Barretenberg proving library, which supports two execution modes: a native binary backend and a WebAssembly (WASM) backend. These two modes execute the same cryptographic prover but in fundamentally different runtime environments, producing meaningfully different performance characteristics. Our framework makes no requirements on whether proofs should be generated locally (native) or in-browser (WASM), so we present benchmarks for both options.

- **Native**: The native backend runs Barretenberg as a platform-specific binary, leveraging native CPU instructions and OS-level threading. This represents the fastest possible proving performance on a given machine.

- **WASM**: The WASM backend runs Barretenberg as WebAssembly, which is how proofs are generated in browser-based applications. WASM execution incurs overhead from the sandboxed runtime, and browser environments introduce additional constraints: Web Worker threading coordination, competition with the DOM for resources, and

tighter memory limits.

Both backends use identical circuit inputs, the same compiled circuit artifacts, and the same measurement methodology: timing witness generation and proof generation as separate phases. This controlled comparison isolates the performance impact of the runtime environment from the circuit complexity itself.

Barretenberg initialization time is the one-time cost of booting the proving runtime and is recorded separately. This cost is negligible for the native backend but averages 1.11 seconds for WASM, as it includes loading and compiling the WASM module. In a browser context, users pay this cost only on their first proof in a browser session. This initialization time is not included in the below benchmark data.

6.3 `MEM` constraint

The membership constraint (`MEM`) proves that an address is in a given set. For example, proving that the sender of this transaction is on an allow-list.

$$C_{\text{mem}} = \text{sender}(\text{txn}) \in p_{\text{allowlist}}$$

where $p_{\text{allowlist}}$ is the public parameter containing the allowed addresses. The circuit for the membership constraint is a single a Merkle proof of a 32 height tree.

Table 6.1: Benchmark results for `MEM` constraint (times in seconds, $n = 10$ runs).

	Native				**WASM**			
	Mean	**Min**	**Max**	**Std Dev**	**Mean**	**Min**	**Max**	**Std Dev**
Witness Generation	0.01	0.00	0.03	0.01	0.01	0.01	0.01	0.00
Proof Generation	0.20	0.18	0.27	0.02	0.39	0.35	0.45	0.03
Total	**0.21**	0.19	0.30	0.03	**0.40**	0.36	0.46	0.03

6.4 `NON-MEM` constraint

The non-membership Constraint (`NON-MEM`) proves that an address is not in a given set. For example, proving that the sender of this transaction is not on a list of sanctioned addresses.

$$C_{\text{non-mem}} = \text{sender}(\text{txn}) \notin p_{\text{sanctioned}}$$

where $p_{\text{sanctioned}}$ is the public parameter containing the sanction list. Similarly to `MEM`, the circuit for `NON-MEM` is 2 merkle proofs of a 32 height tree of adjacent leaves on a sorted merkle tree to prove an address does not exist between them. We see it is takes roughly twice the time of `MEM` because it makes twice the merkle proofs.

Table 6.2: Benchmark results for `NON-MEM` constraint (times in seconds, $n = 10$ runs).

	Native				**WASM**			
	Mean	**Min**	**Max**	**Std Dev**	**Mean**	**Min**	**Max**	**Std Dev**
Witness Generation	0.02	0.02	0.06	0.01	0.02	0.02	0.02	0.00
Proof Generation	0.45	0.42	0.49	0.03	0.83	0.77	0.89	0.03
Total	**0.48**	0.44	0.55	0.03	**0.85**	0.79	0.91	0.03

7. Future Work

Our framework is a start on creating a means to bring compliance and privacy together in a way that meets perceived social needs for both. The components of the framework will benefit from further work addressing both efficiency and breadth of coverage.

7.1 Constraint Domain-Specific Language

To transform written law into a zkSNARK there needs to be a human readable intermediary syntax. The constraint system introduced in 5.4 can be extended into a novel DSL used by regulatory bodies to define compliance. A goal of this DSL is to be easier to read than the formal expression of constraints in raw logic. This would make constraint construction **much** easier for regulators, leading to more framework integration. A larger ecosystem of compliance definitions and a readable compliance syntax would in turn give applications and users more confidence.

This DSL would compile directly into constraints. The addition of this proposed DSL would require no changes to the overall framework and is backward compatible: the framework already is built around the constraint unit.

Example `NON-MEM` constraint translation The non-membership constraint defined as:

$$C_{\text{non-mem}} = \text{sender}(\text{txn}) \notin p_{\text{sanctioned}}$$

Could instead be expressed in a constraint DSL in a more readable fashion:

$$C_{\text{non-mem}} = \texttt{User is not in}\ p_{\text{sanctioned}}$$

Where "is" expresses the temporal context of the current block, and $p_{\text{sanctioned}}$ is the sanction list.

Example `INTERACT` constraint translation The protocol interaction constraint defined as:

$$C_{\text{interact}} = \exists\, \text{t} \in \text{txn_history}(\text{sender}(\text{txn}), [B_0, B_{\text{current}}]) : \text{recipient}(\text{t}) = p_{\text{protocol}}$$

Could be translated into:

$$C_{\text{interact}} = \texttt{User has sent transaction to}\ p_{\text{protocol}}$$

Where "has" means at any point in the user's transaction history and p_{protocol} is the address of the required protocol.

Example published compliance definition Compare the compliance definition $R = C_{\text{non-mem}} \wedge C_{\text{interact}}$ with the current constraint syntax to the same definition using a constraint DSL.

Which is easier to understand?

Current:

$$R = (\text{sender}(\text{txn}) \notin p_{\text{sanctioned}}) \quad \wedge$$
$$(\exists\, \text{t} \in \text{txn_history}(\text{sender}(\text{txn}), [B_0, B_{\text{current}}]) : \text{recipient}(\text{t}) = p_{\text{protocol}})$$

Using constraint DSL:

$$R = (\texttt{User is not in}\ p_{\text{sanctioned}})\ \texttt{AND}\ (\texttt{User has sent transaction to}\ p_{\text{protocol}})$$

7.2 Proof inference and optimization

If several constraints need to be proved, one can just prove their conjunction, but there is likely benefit in transforming a given conjunction of constraints into a shorter and more easily proved form. A similar issue arises with updates to constraints. Say a regulator's compliance definition R_1 is updated to R_2. They might not only publish the full R_2 for new proofs but also $R_{1,2}$ that has the property that $R_1 \wedge R_{1,2} \implies R_2$. Assuming $R_{1,2}$ is less costly to prove than R_2, a user who previously proved R_1 will be rewarded with faster passing of R_2 via the logical implication. Similarly, if certain facts about a

user's transaction history have already been proved, those proofs can be reused. These considerations, at scale, create a need for an indexing of proofs based on what has been proven about what user or transaction history the proof pertains. Such indexing is much more than a simple database-style lookup.

For complex constraint systems, a transaction's proof obligation may consume a significant fraction of overall transaction latency. For such complex constraint systems, any optimization algorithm must take into account not only simplification of statements in boolean logic, but also the estimated cost to generate each component proof. A simpler logic expression might be over a dataset leading to a larger circuit than a less-simple logic expression. The resulting optimization program has features closely related to database query optimization, though with a different cost model. Future work will adapt known optimization techniques from database systems [56] to this novel optimization domain.

Also related to the optimization problem is the underlying execution system. We generate proofs by submitting code in the ZK-friendly domain-specific language Noir [3]. After generating an efficient high-level proof strategy, the actual proofs needed to be coded into Noir in a manner ensuring the fastest possible proof generation by the underlying prover. Aztec, Noir's developer, has published documents on Noir optimization (`https://noir-lang.org/docs/explainers/explainer-writing-noir`), but we anticipate further work to generate optimal Noir code for our framework.

7.3 Backend provers

The framework is designed independent of any ZK prover. By targeting the Noir ZK-DSL, any PLONK-based prover or R1CS-based prover can be used directly. Future work includes broadening our set of provers further, taking into account issues of efficiency and target chains.

7.4 Carry compliance proof through mixers

Commitment tree mixers like Tornado Cash [45] and Privacy Pools [14] unlink the sender and receiver addresses of a transfer, creating privacy for the user. A user enters the mixer by sending funds to the mixer along with a secret. An address can withdraw funds from

the mixer if they provide a ZK proof that know a secret corresponding to a deposit. There is no way to reveal association between any two deposit and withdraw transactions in a mixer without the user revealing their secret. The common use case is a privacy-desiring user withdrawing to an addresses with zero transaction history to obfuscate the association between the two addresses.

Inspired by "Derecho: Privacy Pools with Proof-Carrying Disclosures" [6], the compliance properties required to enter the mixer can be transferred to the withdrawing address by proving the owner of the withdrawing address knows the secret of a leaf in the commitment tree, therefore proving that they have access to an account that met the compliance definition to deposit into the mixer. Then an additional proof needs to be made to assure that the withdrawing account hasn't previously violated compliance constraints. Only the constraints required for depositing to the mixer can be carried to the withdrawing address.

Example: Alice signs up to Coinbase and provides required KYC documents. Alice then receives funds at her on-chain address from a known Coinbase address. Alice can trivially prove her address received funds from Coinbase. Alice does not want her entire transaction history to be public so she deposits some funds in a mixer that has integrated this framework and requires a compliance definition with a single constraint: C_{origin}

$$C_{\text{origin}} = \exists\, t \in \text{first-txn}(\text{sender}(\text{txn}), [B_0, B_{\text{current}}]) : \text{sender}(t) \in A$$

C_{origin} is read as "the sender of this transaction's first transaction was from a set of addresses A", where A is a set of public Coinbase addresses.

Alice proves this to deposit funds to the mixer. She later withdraws her funds from the mixer to a freshly generated address with no transaction history. Even though this new address never received funds from Coinbase, Alice is able to supply her commitment tree secret into a circuit to prove that she is transferring from an address that successfully proved C_{origin}, and therefore should be able to prove C_{origin} when interacting with protocols with her new address.

7.5 Alternative Applications of Constraint Proving

This paper focused on the use case of proving compliance over a transaction history. However, the underlying mechanism of proving arbitrary properties of a transaction history is powerful and can be used in other scenarios. For example, airdrops that only allow addresses with specific properties to claim. It could potentially even move some smart-contract execution off-chain by requiring a proof that an invariant holds for a particular address instead of calculating it on-chain.

7.6 Latency Optimization

Several of the topics in our future work agenda relate to performance. Since proofs in our framework are generated as part of the full execution of a transaction, latency as measured from time of transaction submission to the achievement of a user-acceptable level of finality, is extremely important. Merging the above agendas into a full end-to-end optimization and benchmarking strategy is highly important.

7.7 Industrial and Regulator Acceptance

The ultimate success of our framework rests on its acceptance by regulators, industrial application developers, and users. We have had positive preliminary discussions, but expect that the strongest driver for acceptance will be the competitive ecosystem. Our framework enables stablecoin systems to complete in offering privacy with transparent proof of legality. Regulatory domains seeking to attract financial activity can use our framework to facilitate compliance both in terms of system-development costs and in terms of the cost of compliance verification.

Bibliography

[1] 0xbow. Privacy pools documentation, 2025. https://docs.privacypools.com/.

[2] Aleo Network Foundation. Aleo: The Blockchain Built for Payments. https://aleo.org/, 2025. Accessed: 2025-10-14.

[3] Aztec Team. Introducing Noir: the universal language of zero-knowledge, October 2022. https://medium.com/aztec-protocol/introducing-noir-the-universal-language-of-zero-knowledge-ff43f38d86d9.

[4] S. Baranski, M. Dotan, A. Lotem, and M. Vald. Haze and daze: Compliant privacy mixers. Cryptology ePrint Archive, Paper 2023/1152, 2023.

[5] A. Baumann, M. Peinado, and G. Hunt. Shielding applications from an untrusted cloud with haven. *ACM Trans. Comput. Syst.*, 33(3), Aug. 2015.

[6] J. Beal and B. Fisch. Derecho: Privacy pools with proof-carrying disclosures. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 3197–3211, New York, NY, USA, 2024. Association for Computing Machinery.

[7] M. Ben-Or, O. Goldreich, S. Goldwasser, J. Håstad, J. Kilian, S. Micali, and P. Rogaway. Everything provable is provable in zero-knowledge. In S. Goldwasser, editor, *Advances in Cryptology — CRYPTO' 88.*, Lecture Notes in Computer Science, vol 403. Springer, 1988.

[8] E. Ben Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474, 2014.

[9] E. Ben Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474, 2014.

[10] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, and H. Wu. Zexe: Enabling decentralized private computation. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 947–964, 2020.

[11] J. Burleson, M. Korver, and D. Boneh. Privacy-protecting regulatory solutions using zero-knowledge proofs. Technical report, a16z crypto, 2022.

[12] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform. Web document, 2013. `https://ethereum.org/en/whitepaper/`.

[13] V. Buterin. An incomplete guide to stealth addresses, January 2023.

[14] V. Buterin, J. Illum, M. Nadler, F. Schär, and A. Soleimani. Blockchain privacy and regulatory compliance: Towards a practical equilibrium. *Blockchain: Research and Applications*, 5(1), March 2024.

[15] Chainalysis. Chainalysis sanctions oracle contract. Etherscan, 2022. Ethereum contract address: `0x40C57923924B5c5c5455c48D93317139ADDaC8fb`.

[16] A. Chaudhary and H. Ivey-Law. Sede: Balancing blockchain privacy and regulatory compliance by selective de-anonymization, 2025.

[17] K. Collier. Former twitter employee sentenced to more than three years in prison for spying for saudi arabia. *NBC News*, 12 2022.

[18] R. Daws. FCC fines major telcos for selling users' location data. *Telecoms Tech News*, 4 2024.

[19] N. De. US officials arrest alleged operator of $336m bitcoin mixing service. *CoinDesk*, 2021.

[20] DeFi Llama. Privacy pools protocol analytics, 2025. Accessed: 2025-11-6.

[21] DeFi Llama. Railgun (rail) protocol analytics, 2025. Accessed: 2025-10-27.

[22] DeFi Llama. Tornado cash (torn) protocol analytics, 2025. Accessed: 2025-10-26.

[23] R. Dingledine and N. Mathewson. Anonymity loves company: Usability and the network effect. In *Workshop on the Economics of Information Security*, 2006.

[24] J. Duff and J. Valkema. Ultra Anon: An experimental privacy token with maximum plausible deniability and anonymity set which blurs the lines of public and private state. Eth Denver Hackathon, 2025.
`https://devfolio.co/projects/ultra-anon-9f12`.

[25] Ernst & Young. Blockchain Privacy Technology, 2025. https://blockchain.ey.com/technology.

[26] A. Gabizon, Z. J. Williamson, and O.-M. Ciobotaru. PlonK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *IACR Cryptol. ePrint Arch.*, 2019:953, 2019.

[27] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof-systems. In *Proc. 17th ACM Symp. on the Theory of Computing*, 1985.

[28] J. Groth. On the size of pairing-based non-interactive arguments, 2016. `https://doi.org/10.1007/978-3-662-49896-5_11`.

[29] Hinkal Team. Hinkal Whitepaper: Universal Privacy on Public Chains. https://hinkal-team.gitbook.io/hinkal.

[30] T. E. Jedusor. Mimblewimble, July 2016.

[31] W. Jia, T. Xie, and B. Wang. A privacy-preserving scheme with multi-level regulation compliance for blockchain. *Scientific Reports*, 14(438), 2024.
`https://www.nature.com/articles/s41598-023-50209-x`.

[32] G. Kappos, H. Yousaf, M. Maller, and S. Meiklejohn. An empirical analysis of anonymity in zcash. In *27th USENIX Security Symposium (USENIX Security '18)*, 2018.
`https://arxiv.org/abs/1805.03180`.

[33] keyvank. Eip-7503: Zero-knowledge wormholes - private proof of burn (ppob). Forum post, 2023. https://ethereum-magicians.org/t/eip-7503-zero-knowledge-wormholes-private-proof-of-15456/1.

[34] D. Kuhn. Samourai wallet charges raise existential questions for privacy tech. *CoinDesk*, April 2024.

[35] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari. A survey on the applications of zero-knowledge proofs, 2024. https://arxiv.org/abs/2408.00243.

[36] G. Y. Liao, R. Mayer, A. Soghoian, S. Jain, and E. Tierney. Arc: An open layer-1 blockchain purpose-built for stablecoin finance. Technical report, Circle, August 2025. https://6778953.fs1.hubspotusercontent-na1.net/hubfs/6778953/Arc%20Litepaper%20-%202025.pdf.

[37] G. Maxwell. Coinjoin: Bitcoin privacy for the real world. Bitcoin Forum, August 2013.

[38] C. Moore and S. Ghandi. L2 Ethereum ZK rollup for private and compliant transactions. Whitepaper, 2024.

[39] M. Möser, K. Soska, E. Heilman, K. Lee, H. Heffan, S. Srivastava, K. Hogan, J. Hennessey, A. Miller, A. Narayanan, and N. Christin. An empirical analysis of traceability in the monero blockchain. *Proceedings on Privacy Enhancing Technologies*, 2018(3):143–163, 2018.

[40] M. Nadler and F. Schär. Tornado cash and blockchain privacy: A primer for economists and policymakers. *Federal Reserve Bank of St. Louis REVIEW*, 2023. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4352337.

[41] S. Nakomoto. Bitcoin: A peer-to-peer electronic cash system. Technical report, Bitcoin.org, 2008.

[42] L. H. Newman. Twitter insiders allegedly spied for saudi arabia. *WIRED*, 11 2019.

[43] Paradigm Policy Team. Tradfi tomorrow: Defi and the rise of extensible finance. Policy report, Paradigm, March 2025.

[44] Penumbra Labs. The Penumbra Protocol, 2023.

[45] A. Pertsev, R. Semenov, and R. Storm. Tornado cash privacy solution, 2019. https://berkeley-defi.github.io/assets/material/Tornado%20Cash%20Whitepaper.pdf.

[46] M. Petkus. Why and how zk-SNARK works: Definitive explanation. arXiv 1906.07221v1, 2019.

[47] A. Pfitzmann and M. Hansen. A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management. https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf, 2010.

[48] Polygon Miden. Miden: The edge blockchain. https://miden.com/, 2024. Accessed: 2025-10-14.

[49] K. Post. Exclusive: Massive data leak potentially exposes ukrainian ids to russian intelligence, hackers. *The Kyiv Independent*, 4 2025.

[50] Privacy Patterns. Anonymity set, 2024. https://privacypatterns.org/patterns/Anonymity-set.html.

[51] Railgun. Railgun developer guide, 2024. https://docs.railgun.org/developer-guide/.

[52] D. Rathee, G.-V. Policharla, T. Xie, R. Cottone, and D. X. Song. Zebra: Anonymous credentials with practical on-chain verification and applications to kyc in defi. *IACR Cryptol. ePrint Arch.*, 2022:1286, 2022.

[53] D. Ryan and V. Buterin. EIP-2982: Serenity phase 0. Ethereum Improvement Proposals, September 2020.

[54] J. Sakellariadis and M. Miller. China-linked hackers stole wiretap data from telcos, fbi and cisa say. *POLITICO*, 11 2024.

[55] Secret Network. Secret Network Documentation, 2025. https://docs.scrt.network/secret-network-documentation.

[56] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts, 7th edition*. McGraw Hill Education, New York, NY, 2020.

[57] Solomka I. and Liubinskyi B. Zero-knowledge proof framework for privacy-preserving financial compliance. *Mathematical Modeling and Computing*, 12(1):342–354, 2025. `https://doi.org/10.23939/mmc2025.01.342`.

[58] D. Stone. Webb protocol: A cross-chain private application and governance protocol. Cryptology ePrint Archive, Paper 2023/260, 2023.

[59] Stripe and Paradigm. Tempo: The blockchain designed for payments, 2025. `https://tempo.xyz/`.

[60] M. Sun. U.S. Blacklists Mixer Used to Launder Proceeds from Axie Infinity Crypto Hack, 5 2022. https://www.wsj.com/articles/u-s-blacklists-mixer-used-to-launder-proceeds-from-axie-infinity-crypto-hack-11651873390.

[61] TRM Labs. 2025 crypto crime report: Key trends that shaped the illicit crypto market in 2024. Technical report, TRM Labs, San Francisco, CA, 2025.

[62] United States Congress. Bank secrecy act. 31 U.S.C. § 5318(h), 1970. Requires financial institutions to establish Anti-Money Laundering programs.

[63] U.S. Attorney's Office, Southern District of New York. Tornado cash founders charged with money laundering and sanctions violations. Press Release, August 2023.

[64] N. van Saberhagen. CryptoNote v 2.0, October 2013. Whitepaper.

[65] Y. Wang and J. Gao. A regulation scheme based on the ciphertext-policy hierarchical attribute-based encryption in bitcoin system. *IEEE Access*, 6:16267–16278, 2018.

[66] Wikipedia contributors. Shanghai police database leak. Wikipedia, The Free Encyclopedia, 2025.

[67] Z. J. Williamson. The AZTEC Protocol. Technical report, Aztec, 2018. Whitepaper.

[68] X. Xiong, M. Huth, and W. Knottenbelt. Regkyc: Supporting privacy and compliance enforcement for kyc in blockchains. *IACR Cryptol. ePrint Arch.*, 2025:579, 2025.

[69] zkBob Team. zkbob overview, 2023.

A. Appendix: Framework Specification

Technical specification for framework implementation. This is to be used as a reference sheet for developers integrating this framework. For an example integration, see .

A.1 Terminology

Term	Definition
Constraint	A boolean predicate over blockchain state. The atomic unit of compliance logic.
Compliance Definition	A boolean combination of constraints with a time-validity window. Expressed as a ZK circuit.
ComplianceDefinition Contract	The stable on-chain representation of a compliance definition, published and updated by a regulator. Its address remains constant across updates.
Compliance Version	A snapshot of a compliance definition at a point in time, capturing the verifier, public parameter commitment, time bounds, and artifact references.
Verifier Contract	A smart contract that verifies zero-knowledge proofs. Contains the verification key for a specific circuit.
Public Input	A value visible to both the prover and verifier, passed to the verifier during proof verification.
Private Input	A value known only to the prover (the user), used during proof generation but never revealed.
Content-Addressable Storage	A publicly accessible storage system where data is referenced by a hash of its content (e.g., IPFS, Arweave).
Circuit	A program expressed as a ZK-provable function $f(x, w) \to \{\text{True}, \text{False}\}$, where x is the public input and w is the private input.

A.2 On-Chain Contracts

This section specifies the on-chain contract interfaces that all implementations MUST provide. Interfaces are expressed in Solidity syntax as the canonical reference, but implementations MAY use any smart contract language that provides equivalent functionality.

A.2.1 Chain Requirements

The framework can be deployed on any blockchain that provides:

1. Smart contracts with sufficient expressiveness to implement the interfaces in §A.2.

2. The ability to verify ZK proofs on-chain (either natively or via a precompile/library).

3. A block height or equivalent monotonic ordering for time-bounded validity windows.

A.2.2 IVerifier Interface

Every ZK verifier contract MUST implement the following interface:

```
interface IVerifier {
    function verify(
        bytes calldata _proof,
        bytes32[] calldata _publicInputs
    ) external returns (bool);
}
```

Requirements:

- The `verify` function MUST return `true` if and only if the proof is valid for the given public inputs.
- The `verify` function MAY revert on an invalid proof.
- The serialization format of `_proof` is determined by the proving system used.
- The contents of `_publicInputs` are defined by the circuit. Different circuits MAY have different numbers and types of public inputs.

A.2.3 ComplianceDefinition Contract

The `ComplianceDefinition` contract is the stable on-chain representation of a compliance definition. Applications reference this contract's address, which MUST remain constant across all updates to the compliance definition. Implementations MAY modify the types and arguments associated with the codeboxes below.

ComplianceVersion Struct

Each version of a compliance definition is captured as a `ComplianceVersion`:

```
struct ComplianceVersion {
    address verifier;        // Address of the ZK verifier contract for this
        version
    bytes32 paramsCommitment; // On-chain commitment to public parameters
    uint256 tStart;          // Block height at which this version becomes active
    uint256 tEnd;            // Block height at which this version expires
    string  circuitRef;      // Content-addressable reference to the circuit
        artifacts
    string  paramsRef;       // Content-addressable reference to the full public
        parameters
}
```

Field semantics:

Field	Description
`verifier`	Address of a contract implementing `IVerifier`. Changes on circuit updates; remains the same on parameter-only updates.
`paramsCommitment`	An on-chain commitment to the public parameters used in proof verification. The type of this commitment is circuit-specific (e.g., a Merkle root, a hash). This value MUST be included as a public input during verification so the verifier can confirm the proof was generated against the correct parameter set. Compliance definitions that do not use off-chain public parameters MAY leave this empty.
`tStart`	Block height at which this version becomes active. MAY be set in the future to publish a version before it takes effect. MUST be less than `tEnd`.
`tEnd`	Block height at which this version expires. Set to `type(uint256).max` for no expiration. MUST be greater than `tStart`.
`circuitRef`	A content-addressable identifier (e.g., an IPFS CID) pointing to the circuit artifacts required for proof generation.
`paramsRef`	A content-addressable identifier pointing to the full public parameter data, enabling users to reconstruct proof inputs locally. MAY be empty if the compliance definition has no off-chain public parameters.

Contract Storage

```
ComplianceVersion[] public versions;  // Append-only version history
address public regulator;             // Address authorized to update this
    definition
string public name;                   // Human-readable name
```

The `versions` array MUST be append-only. Implementations MUST NOT modify or delete existing entries.

Contract Functions

```
constructor(address _regulator, string memory _name)
```

Deploys a new compliance definition. The contract is deployed with no versions; the regulator MUST call `updateCircuit` to publish the first version.

```
function verify(bytes calldata proof) external returns (bool)
```

Verifies a user's proof against the currently active version. This is the function that applications call to gate access.

The implementation of `verify` MUST:

1. Retrieve the active version via `getActiveVersion()`.
2. Construct the public inputs array. The contents of this array are circuit-specific. At minimum, implementations SHOULD include:

- A binding to the transaction signer (to prevent proof replay).
- The `paramsCommitment` from the active version (if the circuit uses public parameters).

3. Call `IVerifier(activeVersion.verifier).verify(proof, publicInputs)`.
4. Return the result.

Note on public inputs: The public inputs passed to the verifier are determined by the circuit that the regulator published. The framework does not prescribe a fixed set of public inputs. Regulators define what public inputs their circuit requires, and the `verify` function constructs them accordingly. Common patterns include binding the proof to the prover's address and to the public parameter commitment, but these are not universal requirements.

```
function updateCircuit(
    address newVerifier,
    bytes32 newParamsCommitment,
    uint256 tStart,
    uint256 tEnd,
    string calldata circuitRef,
    string calldata paramsRef
) external
```

Publishes a new version with an updated circuit and verifier. Used when the compliance logic itself changes. MUST be callable only by the `regulator` address. MUST append a new `ComplianceVersion` to the `versions` array.

```
function updateParams(
    bytes32 newParamsCommitment,
    string calldata newParamsRef
) external
```

Publishes a new version with updated public parameters, reusing the current verifier. Used when only parameters change (e.g., adding an address to a sanction list). MUST be callable only by the `regulator` address. MUST append a new `ComplianceVersion` that copies the current version's `verifier`, `tStart`, `tEnd`, and `circuitRef`, replacing only `paramsCommitment` and `paramsRef`.

```
function getActiveVersion() public view returns (ComplianceVersion memory)
```

Returns the most recently published version whose validity window contains the current block height ($t_{\text{start}} \leq$ `block.number` $\leq t_{\text{end}}$). MUST revert if no version is currently active.

```
function getVersionAt(uint256 blockHeight) external view returns (ComplianceVersion
    memory)
```

Returns the most recently published version whose validity window contains the given block height. Enables historical verification. MUST revert if no version was active at the given block height.

```
function getVersionCount() external view returns (uint256)
```

Returns the total number of versions in the history.

A.2.4 Application Integration Pattern

Applications integrate with the framework by referencing one or more `ComplianceDefinition` contracts and gating entry-point functions behind proof verification.

Minimal integration pattern:

```
contract CompliantApplication {
    ComplianceDefinition public complianceDefinition;

    constructor(address _complianceDefinition) {
        complianceDefinition = ComplianceDefinition(_complianceDefinition);
    }

    function protectedAction(bytes calldata proof, /* other args */) external {
        require(complianceDefinition.verify(proof), "Compliance check failed");
        // ... application logic ...
    }
}
```

Requirements:

- Applications MUST call `complianceDefinition.verify(proof, ...)` on entrypoint functions and revert if it returns `false`.
- Applications requiring multiple compliance definitions MUST verify a proof for each.
- The `proof` argument MUST be passed through from the user's transaction calldata.

Multiple compliance example:

```
function protectedAction(
    bytes calldata proofA,
    bytes calldata proofB
) external {
    require(complianceDefinitionA.verify(proofA), "Compliance A failed");
    require(complianceDefinitionB.verify(proofB), "Compliance B failed");
    // ... application logic ...
}
```

A.2.5 On-Chain Commitment to Public Parameters

When a compliance definition uses off-chain public parameters, the `paramsCommitment` field MUST contain an on-chain commitment to those parameters. This commitment serves as a public input during proof verification, binding the proof to the correct parameter set.

The form of the commitment is circuit-specific:

- For set membership/non-membership: a Merkle root of the set.
- For other parameter types: a hash or other binding commitment.

Circuit authors MUST ensure that any off-chain data used in the proof is bound to an on-chain commitment provided as a public input. For example, a Merkle tree stored off-chain is committed to with a Merkle root stored on-chain. The circuit uses the root to verify the validity of the tree the user proved against.

The commitment MUST have the property that a user cannot generate a valid proof against a different parameter set than the one committed to on-chain. The commitment MAY be any type.

A.3 Proof System Requirements

A.3.1 General Requirements

The framework is agnostic to the specific zero-knowledge proving system used. Implementations MAY use any proving system that satisfies:

1. **Soundness**: It MUST be computationally infeasible to generate a valid proof for a false statement.
2. **Zero-knowledge**: The proof MUST NOT reveal the prover's private inputs.

3. **Non-interactivity**: Proofs MUST be non-interactive. The prover generates the proof without communication with the verifier.

4. **On-chain verifiability**: Proofs MUST be verifiable by a smart contract via the `IVerifier` interface.

5. **Succinctness**: The on-chain verification cost MUST NOT grow with the complexity of the compliance definition.

A.3.2 Trusted Setup Considerations

Proof systems that require a per-circuit trusted setup (e.g., Groth16) are permitted but impractical for this framework. Each compliance definition update that changes the underlying circuit would require a new trusted setup ceremony.

Implementations SHOULD use a proof system with one of:

- **No trusted setup**
- **Universal/updatable trusted setup**

A.3.3 Public Inputs

Public inputs are values visible to both the prover and verifier. They are circuit-specific and defined by the regulator who publishes the compliance definition.

Common patterns (RECOMMENDED but not required by the framework):

- **Prover identity binding**: Including the prover's address as a public input prevents proof replay: a proof generated by one user cannot be submitted by another. The mechanism for obtaining the prover's address on-chain is implementation-specific (e.g., `tx.origin`, `msg.sender`, or a signed message).

- **Parameter commitment**: Including the `paramsCommitment` from the active `ComplianceVersion` as a public input binds the proof to the correct public parameter set. This is REQUIRED for any circuit that uses off-chain public parameters, as it is the mechanism by which the verifier confirms that the proof was generated against the authentic parameters.

- **Historical state commitments**: For circuits that make statements over an account's transaction history or other historical data, a public input MUST be provided that allows the verifier to confirm the correctness of the historical data used in the proof (e.g., a state root, a block hash).

A.3.4 Proof Format

The serialization format of proofs is determined by the proving system and MUST match what the deployed `IVerifier` contract expects. The proof is passed as an opaque `bytes` blob through the `ComplianceDefinition.verify()` function to the underlying `IVerifier.verify()` function.

A.3.5 Proof Generation

Proof generation occurs locally on the user's device. The framework does not prescribe whether proofs are generated natively, in a browser (WASM) environment, or remotely using proof generation services. It is important to not rely on secret user inputs if using a remote proving service.

A.3.6 Proof Caching

Users SHOULD cache previously generated proofs locally. A proof remains valid as long as the compliance definition version it was generated against is still active. Implementations SHOULD key cached proofs by the compliance definition contract address and the version count at the time of generation.

A.4 Off-Chain Artifact Storage

A.4.1 General Requirements

Compliance definitions reference off-chain artifacts via content-addressable identifiers stored on-chain in the `ComplianceVersion` struct. Implementations MAY use any content-addressable storage system (IPFS, Arweave, or others) provided that:

1. **Content integrity**: The identifier MUST be derived from the content itself (e.g., a

content hash), ensuring that data retrieved by the identifier matches what was originally published.

2. **Public availability**: Artifacts MUST be publicly retrievable by any user who needs to generate a proof.

3. **Immutability**: Once published, artifacts at a given identifier MUST NOT change. Updates are handled by publishing new versions with new identifiers.

A.4.2 Circuit Artifacts

The `circuitRef` field in `ComplianceVersion` points to the circuit artifacts needed for proof generation. The artifact MUST contain sufficient information for a user to:

1. Determine the circuit's public and private input schema.

2. Generate a proof.

The specific format of circuit artifacts depends on the proving system. For Noir/Barretenberg, it could be a JSON file containing `bytecode` and `abi` fields or the full Noir code for users to compile themselves. Other proving systems will have their own artifact formats.

A.4.3 Public Parameter Data

The `paramsRef` field in `ComplianceVersion` points to the full public parameter data by content id. This allows users to reconstruct whatever data structures are needed for proof generation (e.g., Merkle trees for membership proofs).

The format of public parameter data is circuit-specific. The regulator MUST document the expected format as part of the compliance definition.

If a compliance definition does not use off-chain public parameters, `paramsRef` MAY be empty.

A.4.4 Artifact Availability

If circuit artifacts or public parameter data become unavailable from content-addressable storage, users cannot generate new proofs. Regulators MUST ensure artifacts are pinned or replicated across multiple storage providers for the lifetime of the compliance definition.

A.5 Interaction Protocols

This section specifies the complete lifecycle of a compliance definition, from creation through proof verification.

A.5.1 Regulator: Publish a New Compliance Definition

A regulator publishes a new compliance definition by performing the following steps:

1. **Define constraints**: Express the compliance logic as one or more constraints (§5.4). Document the constraints in their formal notation. The regulator might first want to express constraints in a human-readable format and compile that into constraint logic.

2. **Construct circuit**: Implement the compliance logic as a ZK circuit in the chosen circuit language. The circuit MUST define which inputs are public and which are private. The regulator might use a compiler to generate a circuit given a defined set of constraints.

3. **Compile circuit**: Compile the circuit into the artifact format required by the chosen proving system.

4. **Generate verification key**: Generate the verification key from the compiled circuit.

5. **Generate verifier contract**: Produce a smart contract that implements `IVerifier` (§A.2.2) using the verification key.

6. **Prepare public parameters** (if applicable): Construct the off-chain public parameter data and compute the on-chain commitment (e.g., build a Merkle tree, compute its root).

7. **Upload artifacts to content-addressable storage**:

   - Upload the circuit artifacts → obtain `circuitRef`.
   - Upload the public parameter data (if applicable) → obtain `paramsRef`.

8. **Deploy verifier contract**: Deploy the `IVerifier` implementation on-chain.

9. **Deploy ComplianceDefinition contract**: Deploy a new `ComplianceDefinition` contract with the regulator's address and a human-readable name.

10. **Publish first version**: Call `updateCircuit()` on the deployed contract with:

    - `newVerifier`: address of the deployed verifier contract.
    - `newParamsCommitment`: the on-chain commitment (or empty).
    - `tStart`: block height at which this version becomes active.
    - `tEnd`: block height at which this version expires (or `type(uint256).max`).
    - `circuitRef`: the content-addressable identifier of the circuit artifacts.
    - `paramsRef`: the content-addressable identifier of the public parameters (or empty).

Result: The `ComplianceDefinition` contract is live at a stable address. Applications can reference it. Users can begin generating proofs once `tStart` is reached.

A.5.2 Regulator: Update Public Parameters

When only public parameters change (e.g., adding an address to a sanction list):

1. Prepare updated public parameters and compute the new on-chain commitment.
2. Upload the updated parameter data → obtain `newParamsRef`.
3. Call `updateParams(newParamsCommitment, newParamsRef)` on the existing `ComplianceDefinition` contract.

The circuit and verifier remain unchanged. Users who have cached proofs MUST regenerate proofs for constraints affected by the parameter change, but MAY reuse proofs for unaffected constraints.

A.5.3 Regulator: Update Circuit (Constraint Update)

When the compliance logic itself changes (e.g., adding or removing a constraint):

1. Write, compile, and generate a new verification key for the updated circuit. (Steps 1–4 of §A.5.1)
2. Generate and deploy a new verifier contract.

3. Upload the new circuit artifacts → obtain new `circuitRef`.

4. Prepare updated public parameters if needed → obtain new `paramsRef`.

5. Call `updateCircuit(newVerifier, newParamsCommitment, tStart, tEnd, circuitRef, paramsRef)` on the existing `ComplianceDefinition` contract.

The `ComplianceDefinition` contract address remains stable. Applications require no changes.

A.5.4 Application: Integrate Compliance

An application integrates with the framework by:

1. **Select compliance definitions**: Identify one or more `ComplianceDefinition` contracts that match the application's regulatory requirements.

2. **Store references**: Store the `ComplianceDefinition` contract address(es) in the application contract.

3. **Gate entry points**: Modify entry-point functions to accept a `proof` parameter (one per required compliance definition) and call `complianceDefinition.verify(proof)` before executing application logic. Revert if verification fails.

After integration: All future users of the application are guaranteed to be compliant. The application requires no further action unless it wishes to change which compliance definitions it enforces.

A.5.5 User: Generate and Submit a Compliance Proof

A user interacts with a compliance-requiring application by:

1. **Identify requirements**: Determine which `ComplianceDefinition` contract(s) the application requires. Read the contract address(es) from the application contract.

2. **Read active version**: Call `getActiveVersion()` on each required `ComplianceDefinition` to obtain the `circuitRef`, `paramsRef`, `paramsCommitment`, and `verifier` address.

3. **Check cache**: If a valid cached proof exists for this compliance definition at the current version count, skip to step 7.

4. **Fetch artifacts**: Retrieve the circuit artifacts from content-addressable storage using `circuitRef`. If the circuit uses public parameters, retrieve them using `paramsRef`.

5. **Construct inputs**: Prepare the circuit's public and private inputs:

   - Public inputs as defined by the circuit (e.g., the user's address, the `paramsCommitment`).
   - Private inputs as required (e.g., Merkle proof paths, user secrets).

6. **Generate proof**: Execute the circuit to generate a witness, then produce the ZK proof.

7. **Submit proof**: Call the application's entry-point function, passing the proof(s) as calldata arguments.

8. **Cache proof**: Store the generated proof locally, keyed by the compliance definition address and version count, for future reuse.

B. Appendix: Example Constraints

These example constraints demonstrate the flexibility and expressiveness of our constraint formalism. The constraints below were chosen to highlight some common compliance measures and do not cover all possible constraints. We encourage regulators to be creative when building constraints.

B.1 Non-Membership Constraint (`NON-MEM`)

Proves that an address is not in a given set. For example, proving that the sender of this transaction is not on a list of sanctioned addresses.

$$C_{\text{non-mem}} = \text{sender}(\text{txn}) \notin p_{\text{sanctioned}}$$

where $p_{\text{sanctioned}}$ is the public parameter containing the sanction list.

B.2 Membership Constraint (`MEM`)

Proves that an address is in a given set. For example, proving that the sender of this transaction is on an allow-list.

$$C_{\text{mem}} = \text{sender}(\text{txn}) \in p_{\text{allowlist}}$$

where $p_{\text{allowlist}}$ is the public parameter containing the allowed addresses.

B.3 Protocol Interaction Constraint (`INTERACT`)

Proves that at some point in their transaction history, the sender of this transaction interacted with a specific protocol.

$$C_{\text{interact}} = \exists\, \text{t} \in \text{txn_history}(\text{sender}(\text{txn}), [B_0, B_{\text{current}}]) : \text{recipient}(\text{t}) = p_{\text{protocol}}$$

where p_{protocol} is the address of the required protocol.

B.4 Protocol Avoidance Constraint (`AVOID`)

Proves that throughout their transaction history, the sender of this transaction never interacted with a specific protocol.

$$C_{\text{avoid}} = \forall\, \text{t} \in \text{txn_history}(\text{sender}(\text{txn}), [B_0, B_{\text{current}}]) : \text{recipient}(\text{t}) \neq p_{\text{protocol}}$$

where p_{protocol} is the address of the prohibited protocol.

B.5 Age Constraint (`AGE`)

Requires that an address be at least a certain age. For example, require the sender's first transaction was at least 6 months ago as an effort to reduce sybil addresses.

$$C_{\text{age}} = \exists\, t \in \text{txn_history}(\text{sender}(\text{txn}), [B_0, B_{\text{current}} - p_{\text{min-age}}]) : \text{sender}(t) = \text{sender}(\text{txn})$$

where $p_{\text{min-age}}$ is the minimum required age in blocks. The constraint is satisfied if any transaction from the sender exists before the age threshold, proving the address has been active for at least $p_{\text{min-age}}$ blocks.

B.6 Structuring Constraint (`STRUCTURE`)

Specific example of US Bank Secrecy Act non-structuring requirement. Proves the sender of this transaction has not sent multiple transactions totaling over $10,000 within a window of time to the same recipient.

$$C_{\text{structure}} = \forall\, r \in \text{recipients}(\text{sender}(\text{txn}), [B_j, B_k] \subseteq [B_0, B_{\text{current}}] : (B_k - B_j) \leq p_{\text{window}}) :$$

$$\sum_{\text{txn} \to r} \text{amount}(\text{txn}) < p_{\text{threshold}}$$

where $i = \{[B_j, B_k] \subseteq [B_0, B_{\text{current}}] : (B_k - B_j) \leq p_{\text{window}}\}$ is a state set, p_{window} is the time window (e.g., 24 hours in blocks), and $p_{\text{threshold}} = 10000$ USD equivalent. The state set triggers implicit universal quantification: the predicate must hold for all valid windows.

Vita

TODO